



IRB2002 – Diseño de Sistemas Robóticos
ESCUELA DE INGENIERÍA / MAJOR EN SISTEMAS AUTÓNOMOS Y ROBÓTICOS
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

Informe Final

Diseño de Sistemas Robóticos

Grupo N° 3

Mario Flores
Martín Gallegos
Agustín Mondinelli
Jorge Pérez

12.12.2022

Tabla de Contenidos

1.	Estado final del proyecto	3
1.1.	Descripción del proyecto	3
1.2.	Requerimientos	3
1.3.	Especificaciones	4
1.4.	Normas y Estándares	5
1.5.	Diagrama de Alto Nivel	6
1.6.	Diagrama de Bajo Nivel	7
2.	Descripción de diagrama de bajo nivel	9
2.1.	Alimentación	9
2.2.	Acondicionamiento	9
2.3.	Controlador	10
2.4.	Aislamiento	13
2.5.	Actuadores	14
2.6.	Sensores	17
2.7.	Avances generales	17
3.	Planificación de Costos y Carta Gantt	24
3.1.	Comparación de primera carta Gantt con el desarrollo del proyecto	24
3.2.	Planilla de costos totales	25
3.3.	Planilla de gastos realizados	26
4.	Manual de Usuario	27
4.1.	Hardware y software necesario	27
4.2.	Instalación, ensamblaje y configuración	29
4.3.	Instrucciones de uso	31
5.	Análisis de resultados finales	32
5.1.	Experimentos realizados	32
5.2.	Problemas, fallas críticas y sus causas	33
5.3.	Objetivos cumplidos y no cumplidos	35
5.4.	Mejoras al proyecto	36
6.	Referencias	38
7.	Anexos	39
7.1.	Archivos	39
7.2.	Cálculo de torque de los motores de las ruedas	39
7.3.	Diagrama de tarjetas pre perforadas de interconexión	42



1. Estado final del proyecto

1.1. Descripción del proyecto

En los últimos años, dada la contingencia sanitaria, se ha evidenciado el incremento de residuos quirúrgicos desechados en lugares públicos como calles, parques y playas, y en particular, la contingencia ha llevado al aumento en casi un 9000% los residuos de mascarillas solo entre marzo y octubre del 2020 [1], lo que implica en un riesgo para la salud de las personas que concurren a estos lugares. Por lo tanto, la solución que se propone es un robot capaz de recorrer las playas para recoger y almacenar mascarillas que se encuentran tiradas por donde éste circule. El robot tendrá la capacidad de reconocer lo que es una mascarilla para poder recogerla. Además, otra característica que tendrá es que su movimiento no requerirá de la presencia humana para funcionar y operar. Y, por último, el robot tendrá un sistema de almacenamiento que le permita llevar todas las mascarillas que encuentra, y así poder trasladarse de manera más óptima.

1.2. Requerimientos

Los requerimientos que nos hemos propuesto para que el proyecto cumpla con los objetivos son los siguientes.

- Tiempo de recolección máximo dos veces inferior a un humano promedio.
- Tiempo de autonomía de por lo menos una hora.
- Área de trabajo: $128 m^2$ (área de cancha de voleibol).
- Capacidad de resistencia y adaptación a terrenos adversos (viento, arena y humedad).
- Sistema de detección y recolección de mascarillas.
- Capacidad de reconocer el entorno para recorrer un área delimitada.

Respecto al primer requerimiento, no se logró cumplirlo. Esto se debe principalmente a la manera en que se desarrollaron los movimientos del robot desde que detecta una mascarilla hasta que la recoge, siendo esto un proceso lento. Si se hubiera implementado un control PID más fino y que el movimiento del brazo robótico actuara al mismo tiempo en que el robot se desplaza, entonces se hubiera recolectado mascarillas en un menor tiempo que el obtenido.

Ahora bien, sobre el segundo objetivo, no se ha logrado comprobar que el robot tenga una autonomía de al menos una hora, ya que no se han hecho pruebas durante tanto tiempo sin detenerse. Pero considerando que una batería estaba solamente destinada a alimentar los motores DC y otra batería destinada a alimentar los Servo Motores, entonces estimamos que el robot sí puede cumplir con este requerimiento.



Sobre el área de trabajo que se estableció en $128m^2$, consideramos que el robot es capaz de cumplir con el tercer requerimiento. Ahora bien, se mencionó anteriormente que nunca se han hecho pruebas para verificar si el robot efectivamente tiene una autonomía de al menos una hora, por lo que tampoco se sabe si es capaz de recorrer toda el área sin problemas. Pero sabiendo la capacidad de las baterías, entonces permite pensar que sí cumple con el objetivo.

Respecto al cuarto requerimiento, se ha comprobado que un poco de viento no afecta al funcionamiento ni a los componentes del robot. Por otro lado, el robot nunca fue expuesto a humedad, por lo que no podemos concluir nada respecto a esto. Pero sobre la arena, hemos construido un robot que no es completamente sellado, y por lo tanto durante las pruebas efectuadas, al robot le entró mucha arena donde se encuentran los componentes, aunque ninguno de ellos falló durante las pruebas. Para lograr este requerimiento se debe sellar al robot por completo y que ninguno de los componentes quede expuesto a estar en contacto con la arena.

Por otro lado, sobre el quinto requerimiento, se ha cumplido de buena manera con la implementación del sistema de detección y recolección de mascarillas. Gracias al código desarrollado para segmentar y detectar una mascarilla, y también al código que permite ejecutar los movimientos del brazo robótico, como resultado se obtuvo una segmentación bastante precisa y una fluidez en el movimiento del brazo.

Finalmente, sobre el sexto requerimiento, finalmente se descartó la idea de que el robot sea capaz de reconocer el entorno y sus límites. Lo que se hizo fue implementar un protocolo que recorre un camino definido según la distancia que se traslada. Pero el problema con esto es que finalmente se quemó la ESP32, la cual estaba encargada de leer los encoders de los motores DC, por lo que en el protocolo se cambió distancia recorrida por tiempo de ejecución, lo que no es muy factible en la práctica.

1.3. Especificaciones

Las especificaciones requeridas se pueden clasificar según el área de aplicación.

- **Mecánicas**

- Ruedas con diseño para tracción en arena.
- Material de chasis resistente a la arena y humedad.
- Sistema de pick-up con dos grados de libertad
- Recipiente disponible para almacenamiento de mascarillas con orificios para filtrado de arena.
- Mecanismo accionador capaz de sostener objetos de por lo menos 50g (Peso mascarillas + arena residual).

- **Eléctricas**



- Regulador de voltaje para componentes electrónicos (Sensores y Actuadores).
 - Motores DC con capacidad de torque suficiente para el objetivo propuesto (Dependiendo si son para tracción o para sistema de pick-up, desde 0.504 hasta 2.24 Nm).
 - Sensor de distancia para retroalimentar el sistema de pick-up (Sonar).
 - Fuente de alimentación: 2 baterías de 11.1 V y 2200 mah, considerando 4 motores de tracción de 150 ma, 2 servomotores de 300 ma, y la alimentación de microcontroladores que es de 1300 ma en sus puntos críticos.
 - Sensores de orientación y aceleración para retroalimentar el estado del robot (Giroscopio y acelerómetro).
- **Computacionales**
 - Algoritmo de segmentación de mascarillas mediante procesamiento de imágenes.
 - Protocolo de exploración del área de trabajo.
 - Proceso de filtrado para estimación del estado del robot.
 - Microcontrolador con capacidad de procesamiento y memoria suficiente para cumplir con los algoritmos escogidos.
 - Capturadora de imágenes de resolución de al menos 416 x 416 y un frame rate de por lo menos 2fps.

1.4. Normas y Estándares

A continuación se mencionan los estándares que fueron elegidos para construir el robot.

- ISO 18646-1:2016 [2]: Define métodos y criterios para evaluar el rendimiento de un robot de servicio. Este estándar se enfoca específicamente en el desplazamiento de robots con ruedas. Resulta útil para evaluar el movimiento del robot en desarrollo.
- ISO 18646-3:2021 [3]: Define métodos de evaluación de rendimiento de robots de servicio en la manipulación de objetos. Se evalúan aspectos como: tamaño del manipulador, fuerza de agarre, resistencia al deslizamiento, entre otros. Resulta útil para desarrollar un sistema de pick-up adecuado.
- Certificación IP [4]: Según la numeración de la certificación, se establece el grado de resistencia que tiene un sistema eléctrico a la exposición de polvo y agua. En el caso de este proyecto, la certificación necesaria corresponde a IP66.
- ISO 19649:2017 [5]: Entrega definiciones y vocabulario frecuentemente utilizados en robótica.

Nuestro robot es uno tal que utiliza 4 ruedas para desplazarse. Por lo que, sobre el estándar ISO 18646-1:2016, nuestro robot cumple con las especificaciones, puesto que es capaz de moverse de buena manera y rápida sin presentar problemas.



Por otro lado, consideraos que el robot cumple con el estándar ISO 18646-3:2021, ya que hemos sido capaces de implementar un brazo tal que tiene un tamaño bastante reducido y liviano, la fuerza que realiza es suficiente y la mascarilla no se resbala o cae cuando el gripper logra agarrarla. Considerando todo esto, entonces el sistema de pick-up implementado es adecuado para lo que se necesita.

Ahora bien, sobre la certificación IP66, no se puede concluir nada sobre si el sistema eléctrico resiste a la exposición de agua, ya que no se realizaron pruebas con agua o en un ambiente húmedo. Pero sobre la resistencia al polvo, o en este caso a la arena, si bien el sistema eléctrico estuvo muy expuesto a la arena, pero ninguno de los componentes falló debido a esta situación, por lo que no fue posible verificar si el robot es capaz de cumplir con dicha certificación.

1.5. Diagrama de Alto Nivel

El diagrama de alto nivel final se encuentra en la figura 1, la cual se divide en 6 bloques principales: Fuente de alimentación, acondicionamiento, controlador, sensores, actuadores y aislamiento.

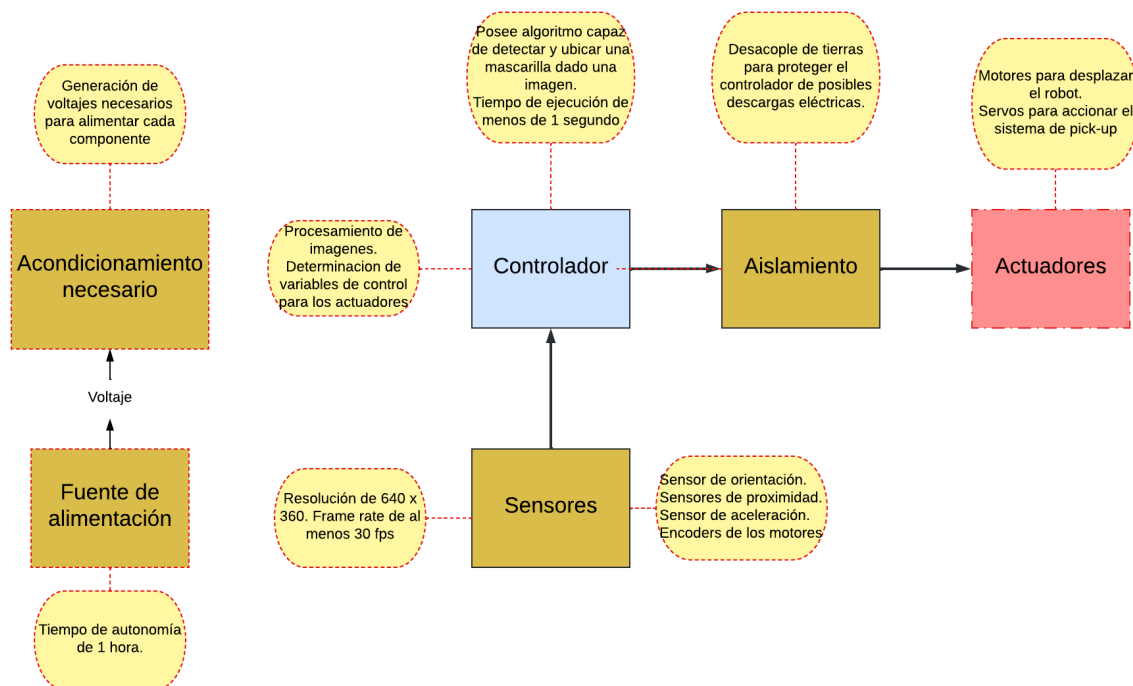


Figura 1: Diagrama de alto nivel



1.6. Diagrama de Bajo Nivel

El diagrama de bajo nivel final se encuentra en las figuras 2, 3, 4, 5 y 6, donde se detallan los componentes específicos que conforman cada bloque del diagrama de alto nivel, también se señalan las características principales de cada uno y como se conectan entre si.

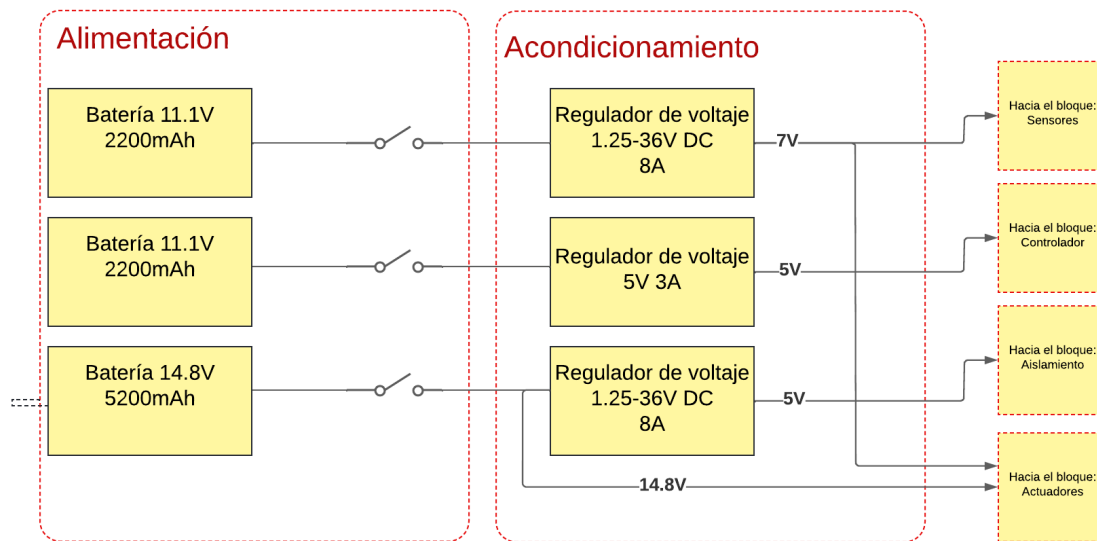


Figura 2: Diagrama de bajo nivel, bloque: Alimentación y acondicionamiento

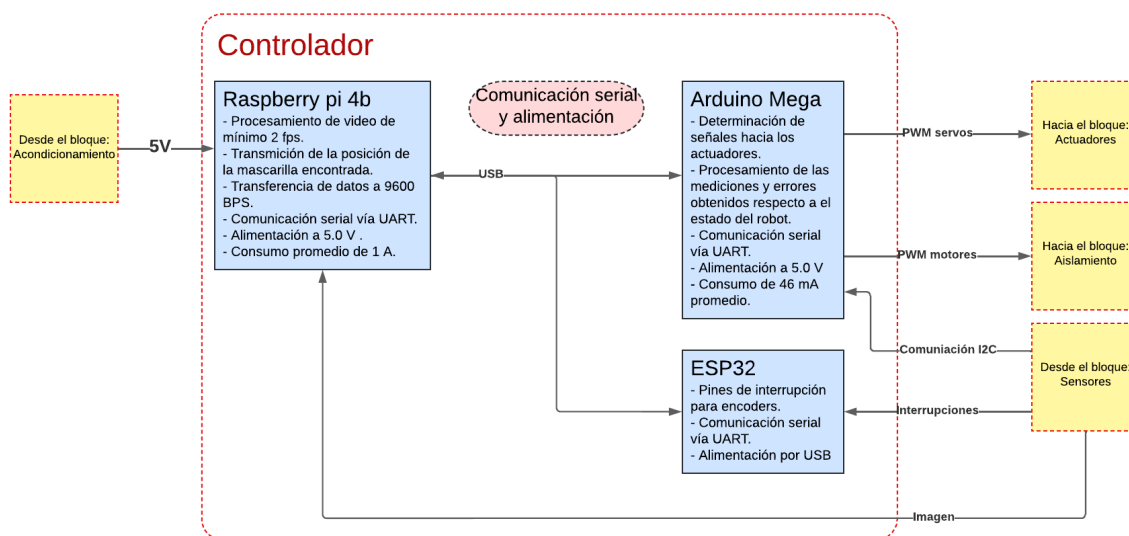


Figura 3: Diagrama de bajo nivel, bloque: Controlador

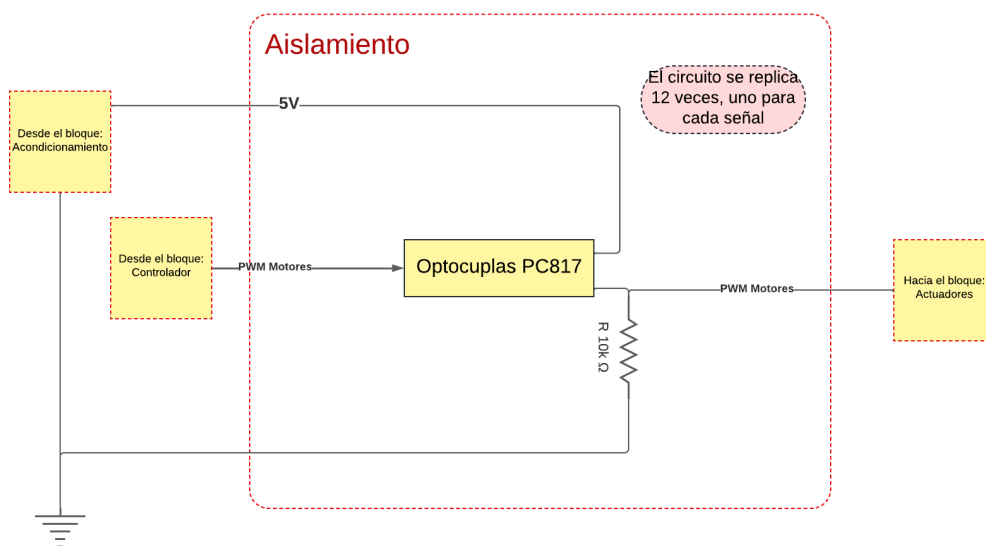


Figura 4: Diagrama de bajo nivel, bloque: Aislamiento

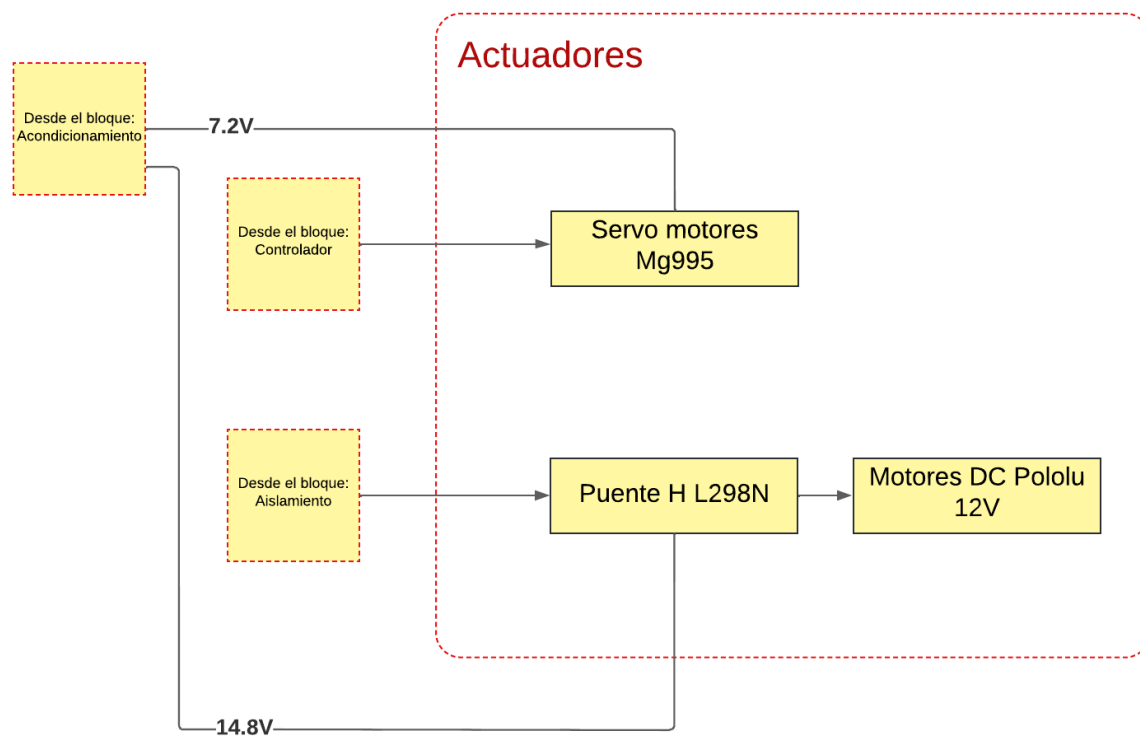


Figura 5: Diagrama de bajo nivel, bloque: Actuadores

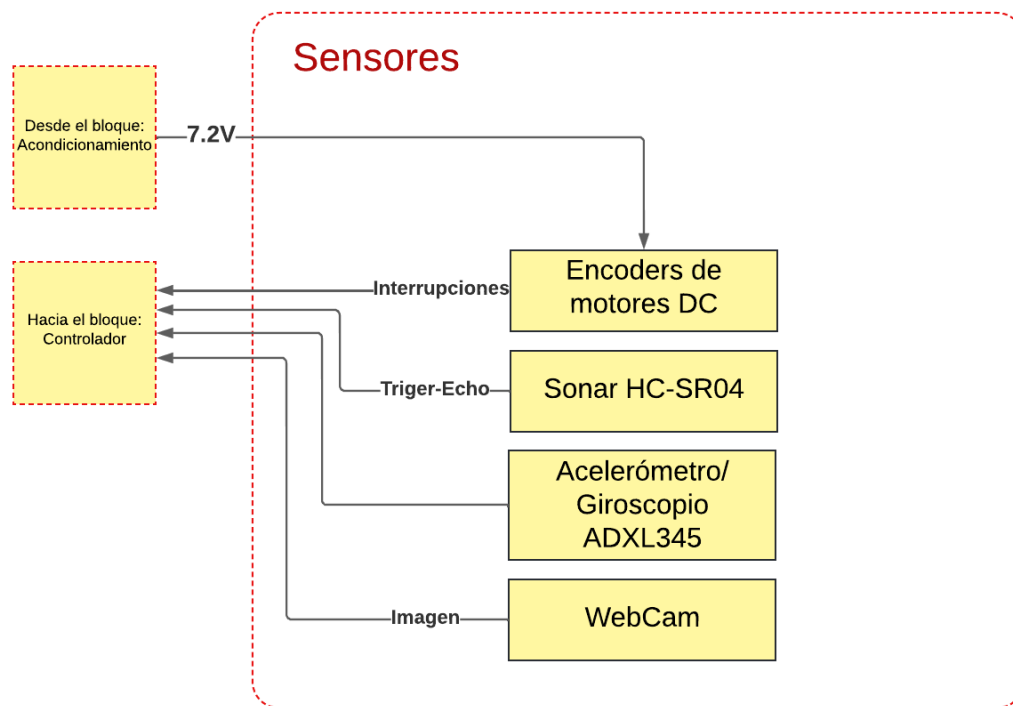


Figura 6: Diagrama de bajo nivel, bloque: Sensores

2. Descripción de diagrama de bajo nivel

2.1. Alimentación

Descripción: El bloque de alimentación tiene como principal función entregar la energía necesaria para hacer funcionar el resto de componentes eléctricos que conforman al sistema. Este bloque se compone de 3 baterías Lipo: 2 de 11.V 2200mAh y uno de 14.8V 5200mAh, este último conecta directamente con los puentes H que alimentan a los motores DC para mover las ruedas. Por otro lado, una de las baterías se dedica exclusivamente para entregar energía a los componentes presentes en el bloque "Controladores", mientras que la batería restante se dedica a entregar energía a servomotores y algunos componentes del bloque "Sensores". Cada batería cuenta con un interruptor para encender o apagar diferentes bloques del sistema.

2.2. Acondicionamiento

Descripción: Este bloque tiene como función de entregar diferentes voltajes a la salida en base al voltaje de entrada proveniente del bloque "Alimentación", ya que los componentes presentes en los otros bloques requieren diferentes voltajes de entrada para funcionar adecuadamente. Este bloque cuenta con 3 reguladores de voltaje, donde dos de ellos corresponden a reguladores de voltaje variable XH-M400, los cuales fueron ajustados para entregar 7.2V y 5V a la salida



respectiva de cada uno, mientras que el tercer regulador corresponde a un regulador de voltaje de 5V 3A.

2.3. Controlador

Descripción: Este bloque corresponde al área encargada de todo el procesamiento de alto nivel y la toma de decisiones del robot, este se compone de 3 elementos: una Raspberry Pi 4b [6], un Arduino Mega [7] y una ESP32[8]. La ESP32 se encarga de leer los datos entregados por los encoders de cada motor DC, de esta forma es posible calcular la velocidad a la que se desplaza el robot. Por otro lado, el Arduino Mega se encarga de leer los datos provenientes del sonar y del acelerómetro/giroscopio, mientras que envía señales de control a los motores del bloque "Actuadores". Finalmente, la Raspberry Pi tiene como función leer las imágenes registradas por la cámara web y enviar instrucciones al Arduino Mega en base a la información que entrega la imagen.

Diagramas de flujo: A continuación, se describe la lógica detrás del funcionamiento de cada elemento presente en este bloque:

- **ESP32:** El funcionamiento de este componente es simple, en cada ciclo del código, la ESP32 lee la información entregada por los encoders de los motores, estima la velocidad de cada rueda en revoluciones por minuto (RPM) y envía las velocidades calculadas a la Raspberry Pi mediante comunicación serial.
- **Arduino Mega:** El funcionamiento de este componente se resume de la siguiente forma: en primer lugar, el Arduino inicializa y calibra todos sensores conectados a él. Luego, el Arduino entra en un estado de espera, el cual permanece hasta registrar una instrucción entregada por la Raspberry Pi mediante comunicación serial. Luego de recibido correctamente el mensaje con la instrucción, el Arduino decodifica la información entregada y actúa en base a ella. Dentro de las instrucciones que recibe este componente se encuentran: un mensaje de STOP que le indica al Arduino detener todos sus motores, instrucciones para manejar la velocidad de los motores y un mensaje de PICK UP que indica que el Arduino debe ejecutar una secuencia de comandos a los servo motores para hacer que el brazo robótico se mueva y realice el gesto de recoger una mascarilla.
- **Raspberry Pi:** El funcionamiento de este elemento se divide en 2 estados principales: Exploración y Recolección, donde este último se subdivide en: Orientación, Acercamiento y Pick-UP. En el estado de exploración, la Raspberry Pi recibe la información proveniente del acelerómetro/giroscopio conectado al Arduino Mega, y en base a ello implementa un algoritmo de seguimiento de trayectorias, donde se utiliza un controlador PID que utiliza como función de error el error entre el ángulo actual medido y un ángulo de referencia, de modo que el robot se mantenga en una dirección en línea recta. De esta forma, es posible programar que el robot siga una trayectoria en particular solamente modificando el ángulo de referencia dependiendo de la curva que se desee formar. Mientras el robot sigue la trayectoria establecida, se leen constantemente las imágenes capturadas por la cámara, para posteriormente ser procesadas en un algoritmo de segmentación de mascarillas, el cual consiste en un algoritmo de segmentación HSV que ha sido calibrado para segmentar objetos de tonalidades azules-celestes, dado que un alto porcentaje de las mascarillas



utilizadas actualmente presentan este color. En el caso en que la segmentación logra detectar un objeto, el robot cambia su estado de Exploración a Orientación. En este estado, el robot calcula el centro de masa del objeto detectado, para luego implementar un control PID basado en orientación, donde la función de error corresponde a la desviación del centro de masa con respecto al centro de la cámara, de esta forma, el robot rota hasta alinearse completamente con la mascarilla detectada. Cuando el error es lo suficientemente pequeño, el robot cambia su estado a Acercamiento, donde este utiliza un control PID basado en distancia para aproximarse a la mascarilla y localizarse a una distancia determinada, de modo que se encuentre a rango para poder ser recogida. Cuando el robot alcanza dicha distancia, se pasa inmediatamente al estado Pick-Up, donde la Raspberry Pi envía la instrucción de ejecutar la secuencia de movimientos del brazo robótico para recoger la mascarilla. Una vez finalizado el proceso, el programa vuelve al estado de Exploración. Para evitar colisiones con objetos en movimiento, se lee constantemente los valores entregados por el sonar ubicado en el brazo del robot, donde se enviará una señal de STOP en el caso de que se registre un objeto demasiado cerca.

Dado que el robot no funcionó correctamente en la presentación final, es necesario aclarar los puntos que no lograron ser implementados de forma adecuada, donde, en pocas palabras, se logró implementar los diferentes estados de forma independiente, pero no se pudieron integrar adecuadamente para formar el flujo principal. Tampoco se pudo comprobar que la sección de freno de emergencia se implementó correctamente.

Los diagramas de flujo se presentan a continuación:

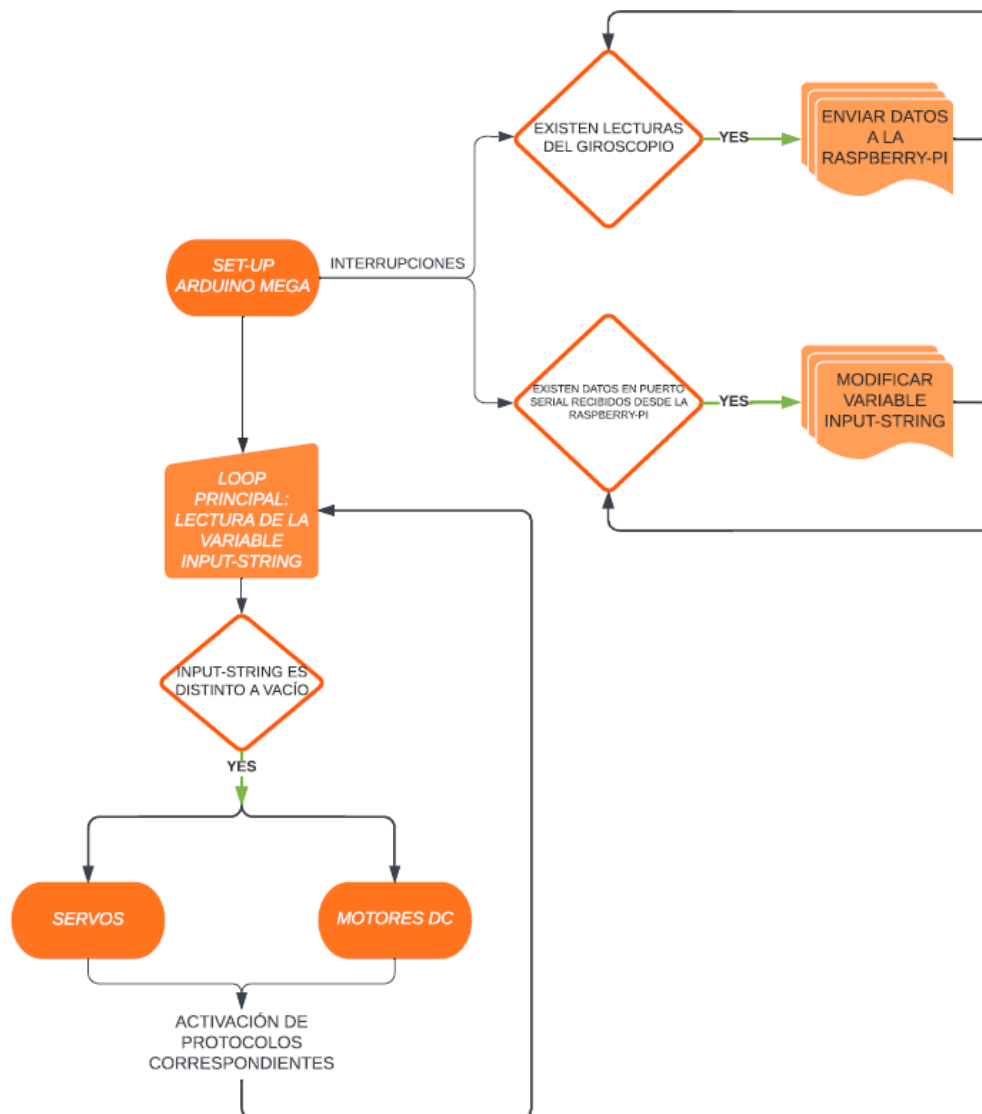


Figura 7: Diagrama de flujo del funcionamiento de Arduino Mega

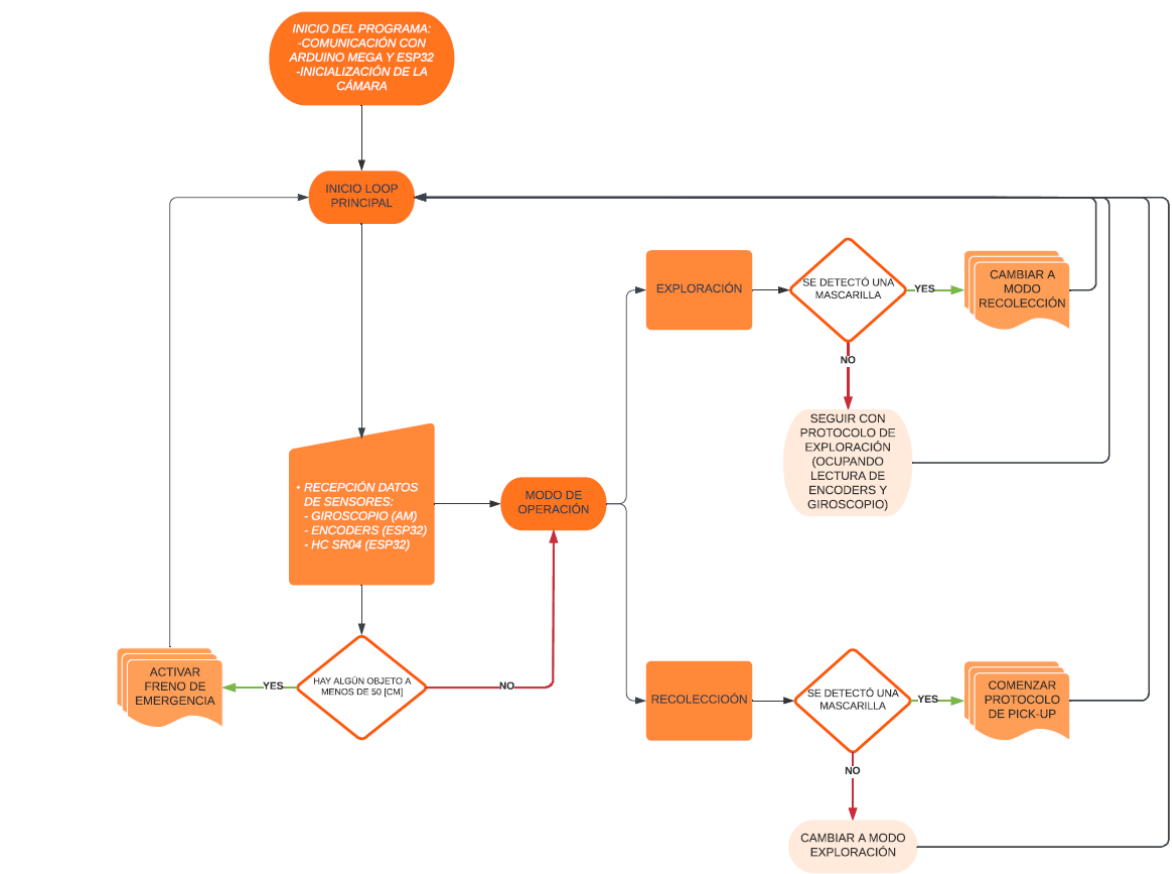


Figura 8: Diagrama de flujo del funcionamiento de Raspberry Pi

2.4. Aislamiento

Descripción: Este corresponde a un bloque nuevo considerando la versión anterior del diagrama. Este tiene como función desacoplar las señales generadas por el Arduino y entregarlas a los puentes H que controlan los motores, esto se hace con el fin de proteger los componentes vitales del sistema ante cualquier falla o peak de corriente generado en los puentes H. El bloque consiste en 12 optocouplers PC817[9], los cuales consisten en un led infrarrojo conectado a cada señal proveniente del Arduino, dicha señal excita al led, lo que hace encenderse o apagarse según corresponda, la luz generada es captada por un foto-transistor que circula corriente según la luz que recibe, el pin negativo es conectado a una resistencia de pull-down y luego a la tierra respectiva del puente H, este pin también conecta con las conexiones respectivas de los puentes H y el pin positivo se conecta a una alimentación de 5V. De esta forma, las señales son transmitidas al puente H sin tener una conexión directa con el Arduino.

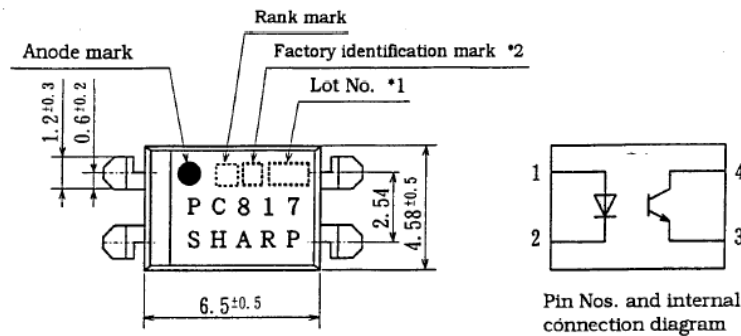


Figura 9: Optocupla PC817 [9]

2.5. Actuadores

Descripción: Este bloque contiene todos los elementos necesarios para que el robot se desplace e interactúe con su entorno. En particular, este bloque se compone de 4 motores DC Pololu 37D de 12V[10] y 4 servo motores Mg995[11]. Para controlar las velocidades de cada motor, se utilizan 2 puentes H L298N[12]. En el caso de los motores DC, estos se encuentran ensamblados a su respectivo soporte en forma de L, el que a su vez se encuentra ensamblado a una plancha de madera MDF de 6mm de espesor. Por otro lado, los servo motores se encuentran ensamblados a un brazo robótico formado por piezas fabricadas con impresoras 3D.

Cálculos : En el anexo 7.2 de este informe se encuentran los cálculos efectuados para determinar los torques y velocidades angulares necesarios para desarrollar el robot.

Planos mecánicos:

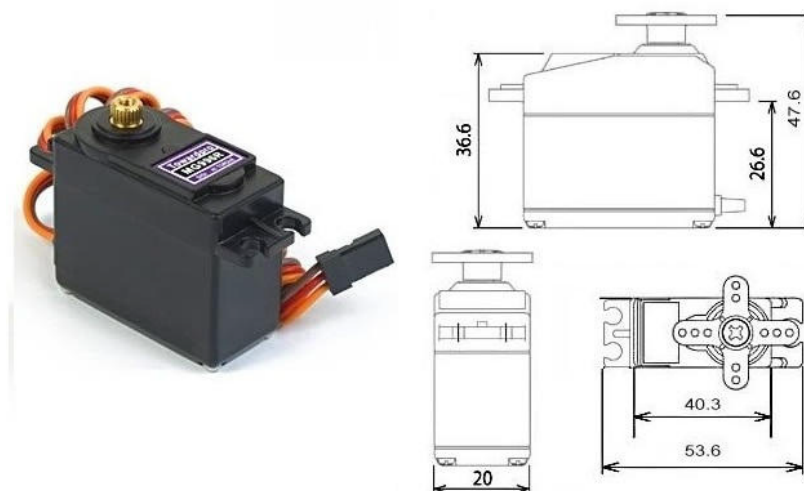


Figura 10: Servo motor Mg995 [11]

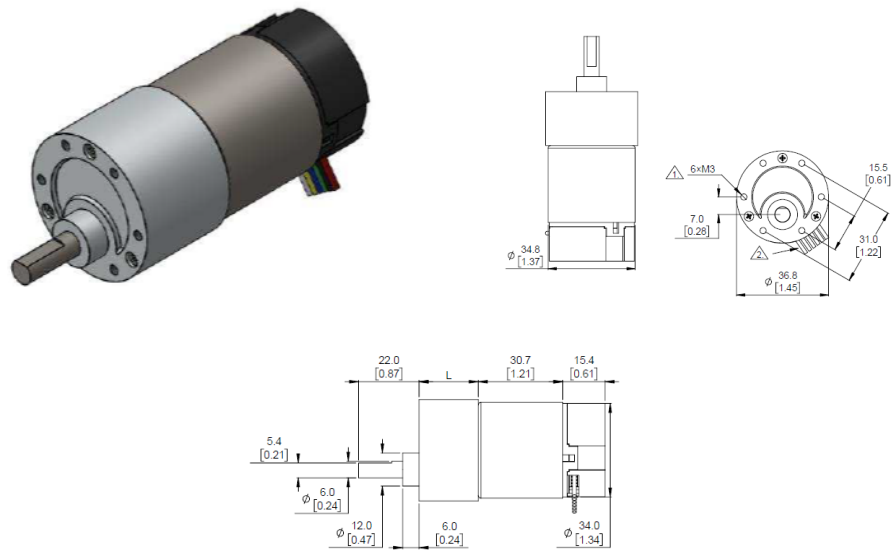


Figura 11: Motor DC pololu 37D [10]

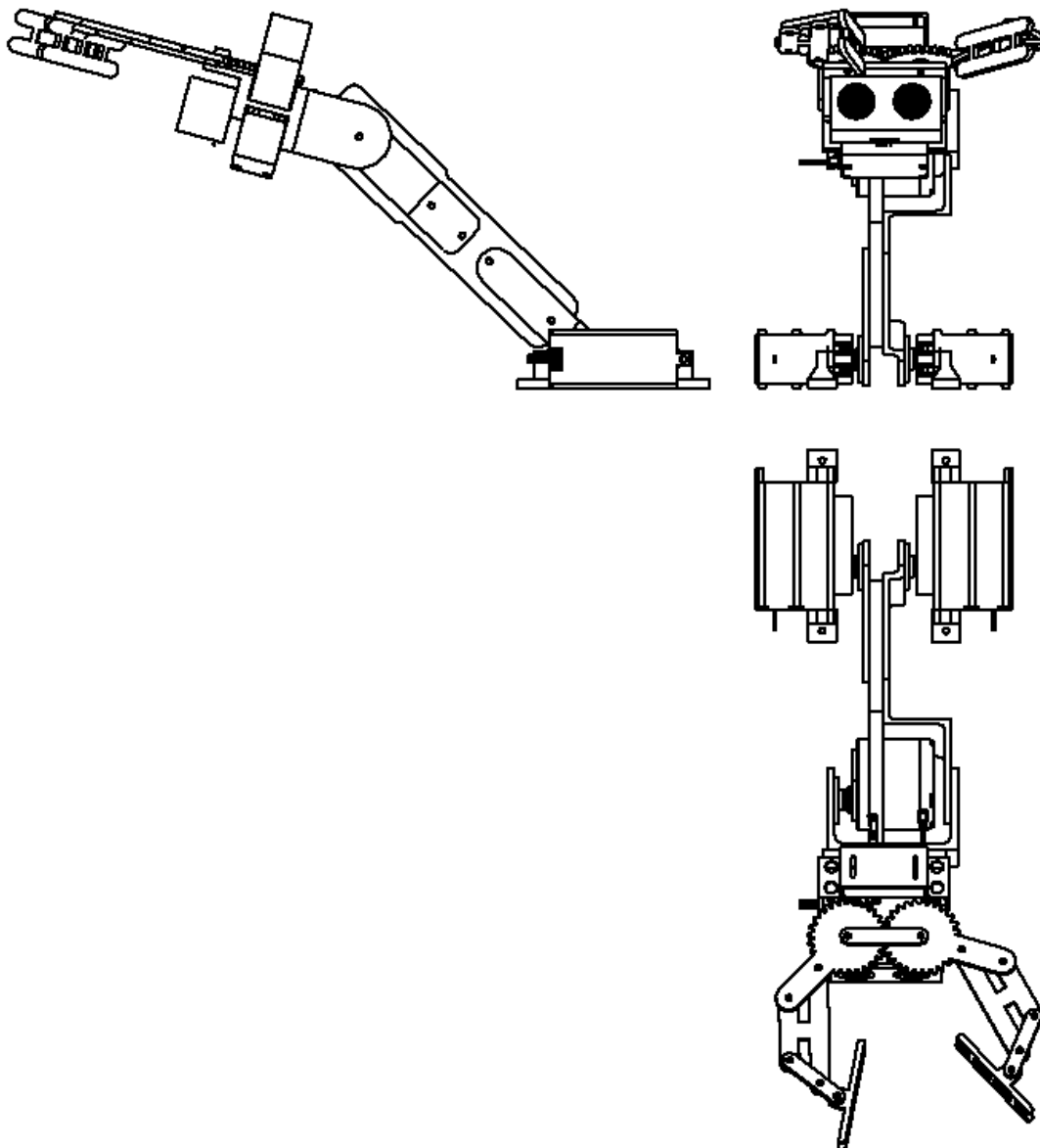


Figura 12: Vistas de ensamble de brazo robótico

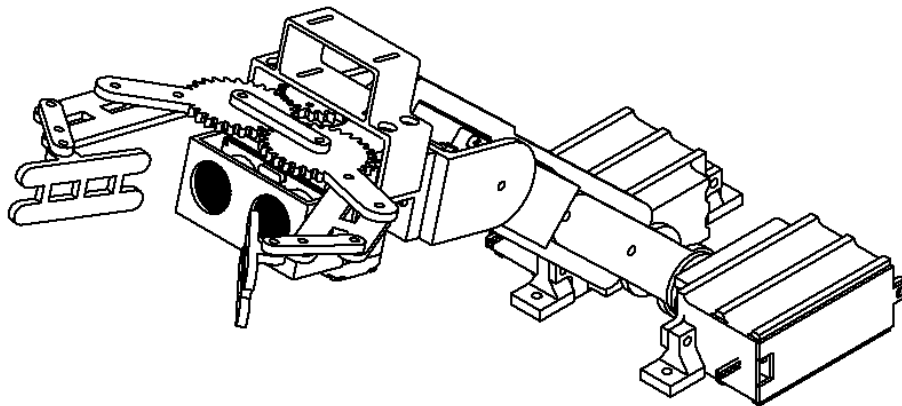


Figura 13: Vista isométrica de ensamblaje de brazo robótico

2.6. Sensores

Descripción: Este bloque reúne todos los dispositivos encargados de entregar información del entorno al controlador. Se compone de 4 encoders provenientes de los motores DC Pololu, los cuales registran la velocidad a la que gira cada eje según un tren de pulsos generados por un sensor Hall, una cámara web, la cual captura imágenes de 640x320 de resolución en tiempo real, un sonar HC-SR04, el cual registra la distancia de objetos en base a tiempos de emisión y recepción de ultra sonidos, y un acelerómetro/giroscopio MPU6050[13], el cual, a grandes rasgos, funciona con un sistema de masa resorte (Giroscopio) que entrega velocidades angulares y un sistema piezoeléctrico (Acelerómetro) que transforma las aceleraciones en señales de voltaje, el cual a través de ambas mediciones es capaz de entregar ángulos respecto a un marco de referencia dado al iniciar el sensor.

2.7. Avances generales

Fuera del avance realizado en los bloques anteriormente mencionados, se realizaron avances que engloban diferentes bloques, los cuales se describen a continuación:

Interconexión de bloques

Descripción: Para realizar la conexión física de los diferentes bloques del diagrama de bajo nivel, se diseñaron y fabricaron tarjetas PCB, los cuales cuentan con los tipos de conexión necesarios para cada componente. Además, las conexiones fueron distribuidas espacialmente, de forma tal que las conexiones lógicas se encuentren lo más distanciadas posibles de las conexiones



de transmisión de potencia, con el fin de minimizar la probabilidad de que algún componente sensible resulte dañado producto de una falla eléctrica.

Diseño de PCB: Los diseños se encuentran adjuntados en el anexo 7.3 de este informe

Protecciones de componentes críticos

Descripción: Dado que el ambiente de trabajo del robot propuesto corresponde a una playa, este se verá expuesto a humedad y partículas de arena que pueden ingresar y dañar componentes tanto eléctricos como mecánicos. Por esta razón, se diseñaron e imprimieron piezas que tienen como finalidad cubrir la mayoría de componentes ante el ingreso de arena.

Planos mecánicos: Las piezas que fueron diseñadas se adjuntaron junto al presente informe. En el caso de la Raspberry Pi, se optó por una carcasa de metal.

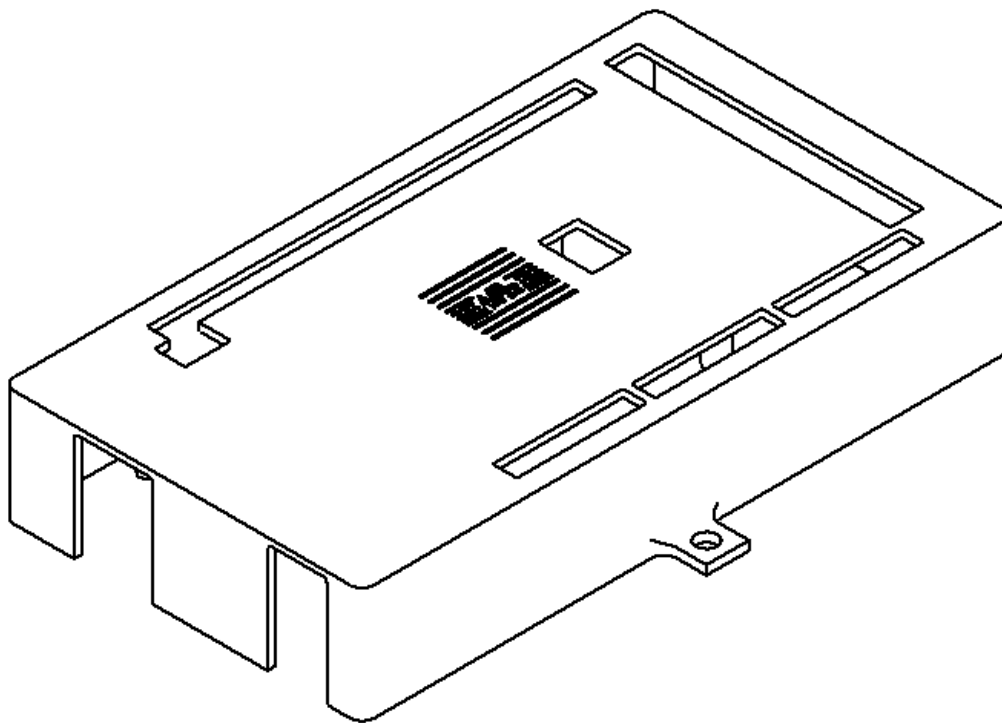


Figura 14: Protección Arduino Mega

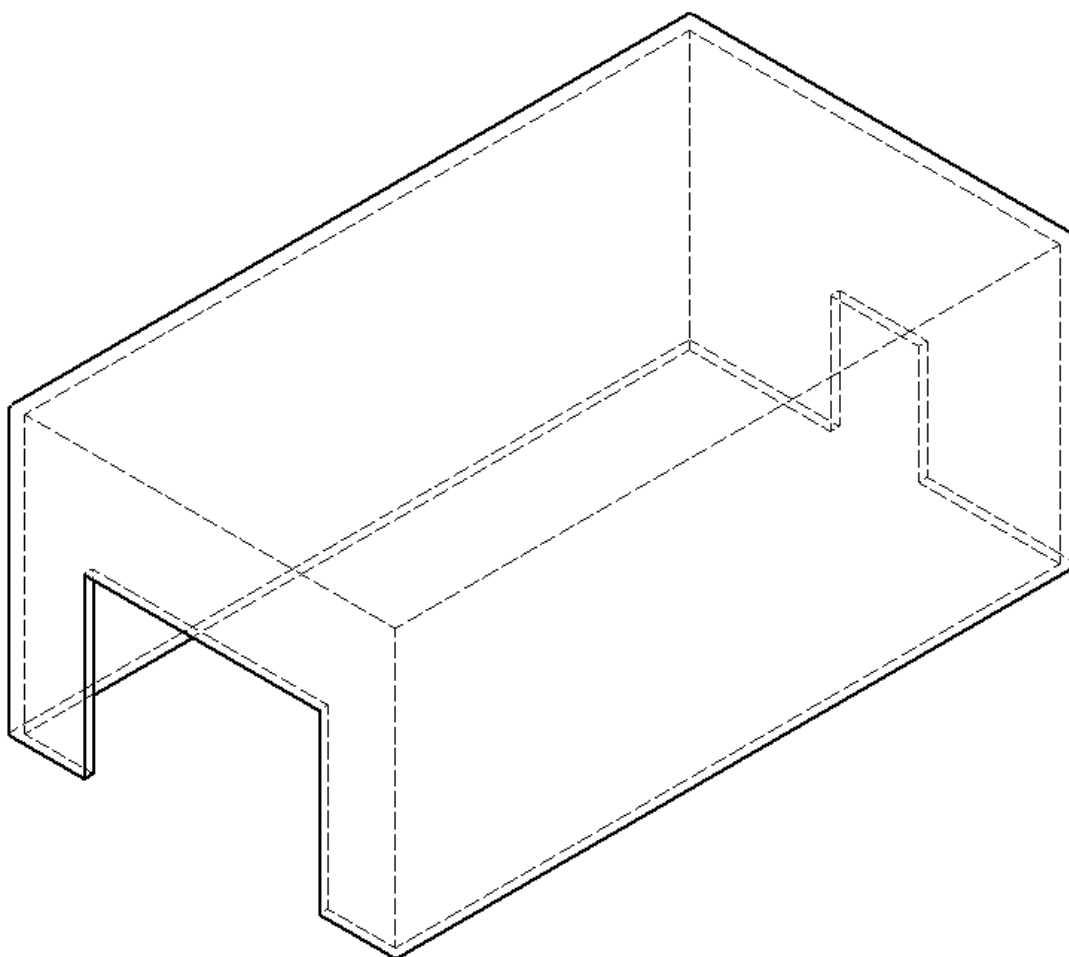


Figura 15: Protección ESP32

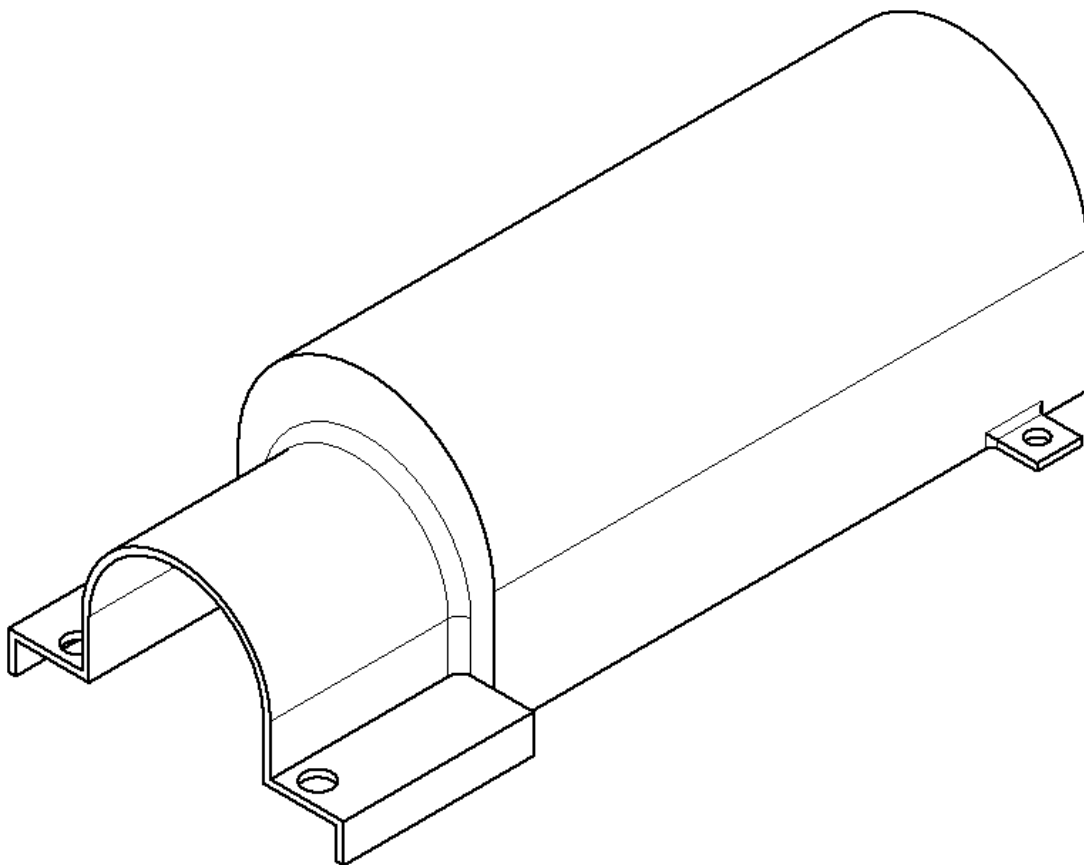


Figura 16: Protección de motores DC

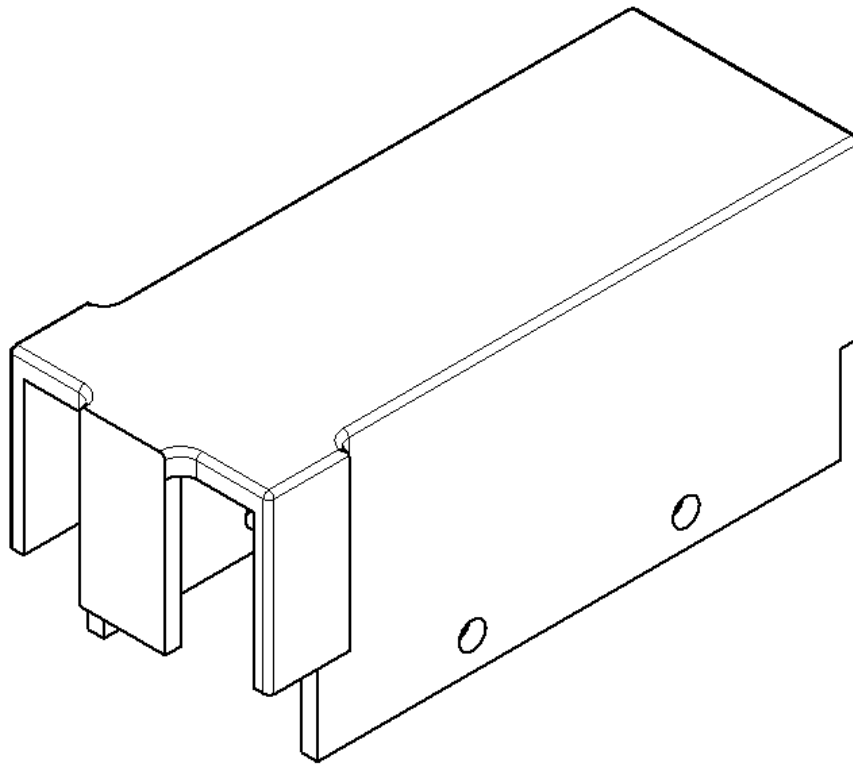


Figura 17: Protección de baterías Lipo 11.1V

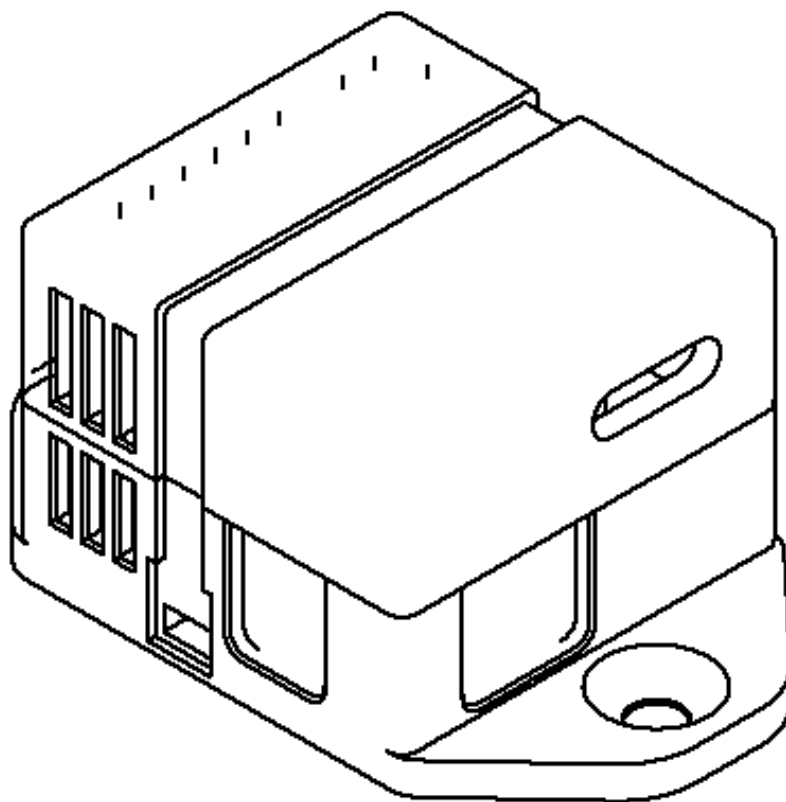


Figura 18: Protección de puente H. [LINK](#)

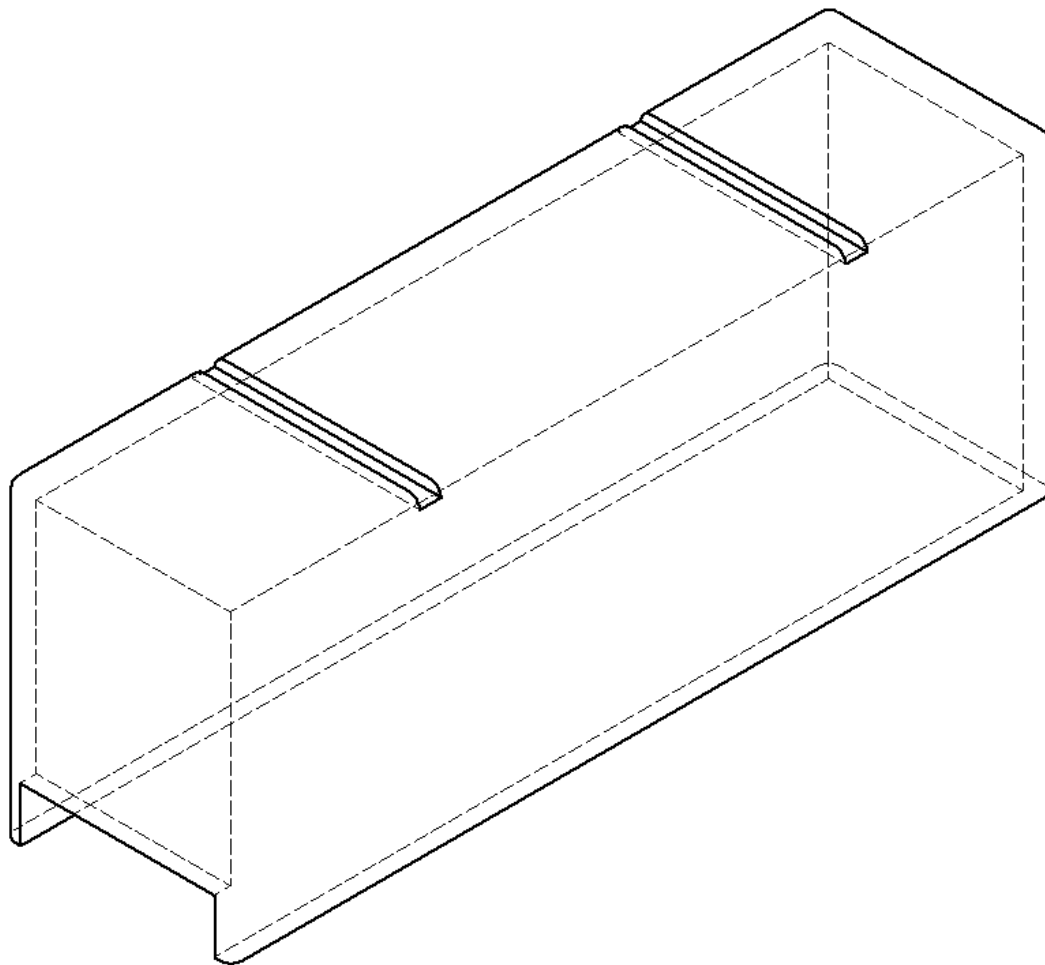


Figura 19: Protección de baterías Lipo 14.8V

Pruebas efectuadas: Para confirmar que las piezas diseñadas cumplen adecuadamente con su función, se realizó una prueba en terreno, donde se colocó el robot con sus respectivas protecciones, y se le dio la instrucción de desplazarse en la arena. Con esta prueba, se obtuvo como resultado que las protecciones lo gran evitar la filtración de arena al robot de forma adecuada. Sin embargo, dicha prueba no considera la posibilidad de que el robot se exponga a humedad, por lo que no se obtuvo información al respecto. Una observación importante es que, en el diseño final, existen componentes que no cuentan con su respectiva protección contra la arena, lo cual es un fallo grave en el diseño.

En conclusión, las protecciones diseñadas cumplen parcialmente uno de los objetivos propuestos, que corresponde al de que se debe ser resistente a la humedad y al polvo. Por otro lado, queda como aspectos a mejorar la incorporación de más protecciones para el resto del sistema.



3. Planificación de Costos y Carta Gantt

3.1. Comparación de primera carta Gantt con el desarrollo del proyecto

A continuación, se presenta la primera carta Gantt, la cual fue propuesta a principios de semestre:

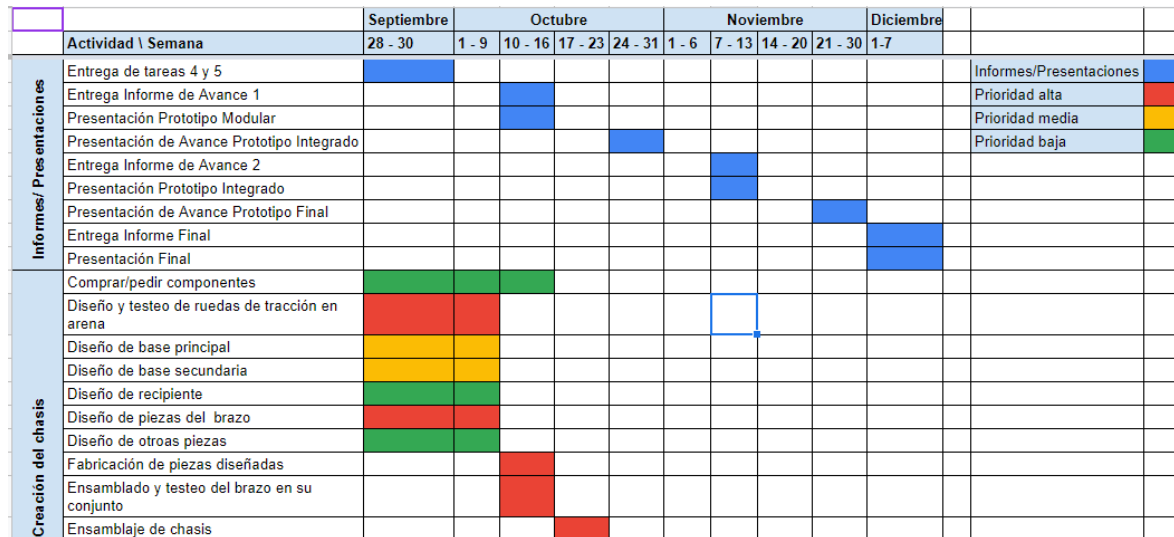


Figura 20: Carta Gantt, parte 1

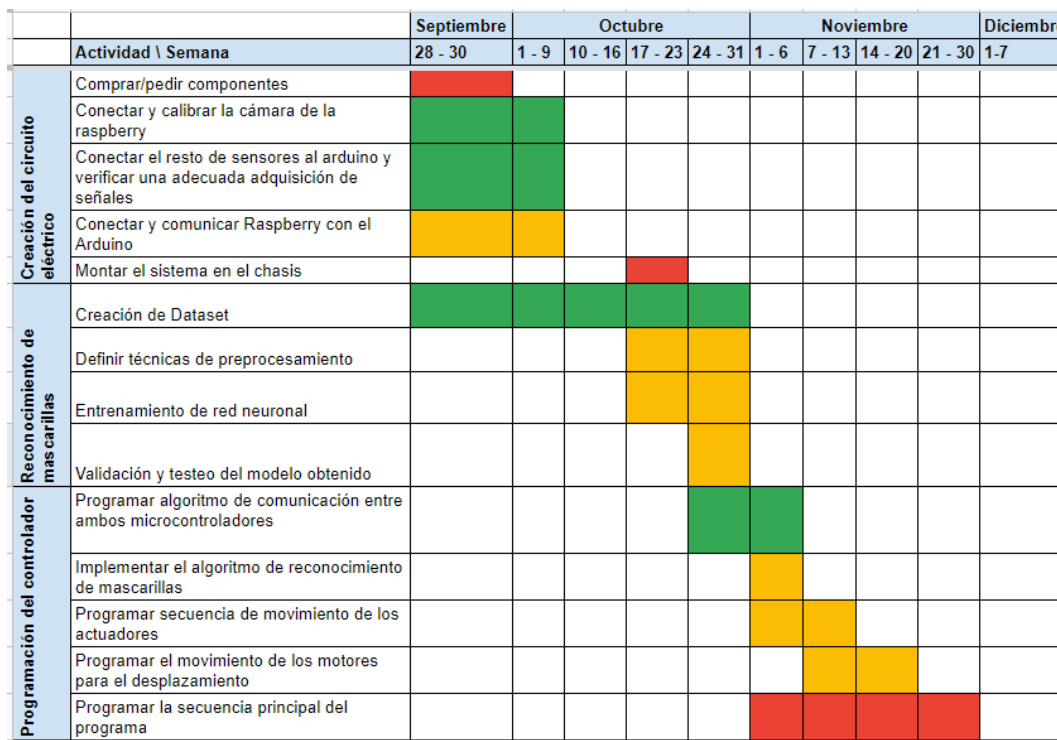


Figura 21: Carta Gantt, parte 2



Al comparar esta carta Gantt con el desarrollo real del proyecto, resalta la siguientes discordancias:

En primer lugar, una de las principales diferencias entre el desarrollo real y el propuesto, fue el diseño y testeo de las ruedas en arena, dado que solo se realizó una prueba dentro de los primeros meses, donde se utilizó una versión incompleta del robot para verificar si el diseño propuesto logra hacer tracción en arena. Luego, se realizó otra prueba en los últimos días con la versión final, en los que se pudo observar que el robot presenta ciertas dificultades en su movimiento, lo que dio como resultado que, en la presentación final, el robot no haya sido capaz de desplazarse de forma adecuada en arena.

Por otro lado, otra discordancia corresponde a que, en la versión inicial de la carta Gantt, no se consideró el diseño de tarjetas PCB dentro de los plazos disponibles, esto dio como resultados que el desarrollo se atrasara por lo menos por dos semanas, donde dichas semanas se dedicaron exclusivamente al diseño de las tarjetas PCB. La gran extensión de esta etapa de diseño se debe principalmente a la falta de conocimiento que se tenía como grupo en cuanto al diseño de PCB. Inicialmente, se consideró realizar un diseño de PCB propio, para luego ser fabricado la ruteadora que se encuentra en el departamento de ingeniería eléctrica de la universidad. Sin embargo, dicha ruteadora había sufrido un desperfecto, por lo que no estaba disponible. Como plan de contingencia, se utilizaron tarjetas pre-perforadas, las cuales tuvieron que ser construidas manualmente, es por esta razón que el atraso en el resto del proyecto fue tan severo.

Otra discordancia importante corresponde a la sección de creación del programa principal y el diseño del algoritmo de detección de mascarillas, este último sufrió grandes modificaciones a lo largo del tiempo. Originalmente, se había diseñado esta última sección pensando en que para la detección de mascarillas se utilizaría una red neuronal entrenada con un dataset creado con datos propios, pero esta idea tuvo que ser desechada principalmente por dos razones: primero, la capacidad de procesamiento de la Raspberry Pi disponible era bastante limitada, lo que provocaba que el programa se ejecute demasiado lento, segundo, se pudo notar que un algoritmo de segmentación basado en colores funcionaba con gran eficiencia. Por otro lado, el tiempo considerado para realizar el programa principal que integraba todos los bloques planteados en el diagrama de bajo nivel fue mucho menor al esperado, debido principalmente a los atrasos que se presentaron en los pasos anteriores, por lo que no se pudo desarrollar el programa principal completamente, y no se lograron corregir a tiempo los errores que surgieron mientras se programaba.

3.2. Planilla de costos totales

Las siguientes tablas detallan los costos de cada componente que compone el proyecto desarrollado, donde el costo total corresponde a \$ 762.043,00 pesos chilenos:



Producto	Tienda	País	Precio unitario	Unidades	Total
Madera MDF	Sodimac	Chile	9.990,00	1	9.990,00
Acoples rígidos (diferentes dimensiones)	Cimech3D	Chile	2.290,00	8	18.320,00
Ruedas Juguete Monster Truck para arena		Chile	20.000,00	1	20.000,00
Eje con hilo	MercadoLibre		1.990,00	1	1.990,00
Eje de 8mm de diámetro y 1m de longitud	Cimech3D	Chile	12.990,00	1	12.990,00
1 Kg de material de impresión 3D PLA	Cicla3D	Chile	11.750,00	2	23.500,00
Basurero	Ikea	Chile	3.990,00	1	3.990,00
Tornillos y tuercas de diferentes diámetros		Chile	7.990,00	1	7.990,00
Acoples tipo L para motores DC pololu	Pololu	Estados unidos	8.520,00	4	34.080,00
Soportes con rodamiento	Cimech3D	Chile	3.290,00	4	13.160,00
				Total	146.010,00

Figura 22: Costos totales, parte mecánica

Producto	Tienda	País	Precio unitario	Unidades	Total
ESP32 dev kit	Mouser	Chile	10.927,00	1	10.927,00
Arduino MEGA	Arduino	Chile	52.990,00	1	52.990,00
Raspberry Pi 4b 2GB	PcFactory	Chile	94.690,00	1	94.690,00
Optocouplars PC817	Afel	Chile	2.990,00	6	17.940,00
Placas preperforadas	Afel	Chile	1.200,00	6	7.200,00
Pack de 10 resistencias de 10K Ohm	Altronics	Chile	400,00	1	400,00
Pin header 40 pines (Macho/Hembra)	MCI	Chile	250,00	4	1.000,00
Regulador de Voltaje Step Down XH-M400	Afel	Chile	5.000,00	2	10.000,00
Puente H L298N	Afel	Chile	3.000,00	2	6.000,00
Motor DC Pololu 37D	Pololu	Estados Unidos	44.484,00	4	177.936,00
Servo-motor Mg995	Afel	Chile	7.000,00	4	28.000,00
Acelerómetro-giroscopio ADXL345	Afel	Chile	2.300,00	1	2.300,00
Batería Lipo 11.1V 2200 mAh	BateríasLipo	Chile	29.990,00	2	59.980,00
Batería Lipo 14.8V 5200 mAh	XRC Hobbies	Chile	79.990,00	1	79.990,00
Interruptores 2p 21x15mm	MercadoLibre	Chile	698,00	3	2.094,00
Conector XT60/XT60U-F	MercadoLibre	Chile	2.500,00	3	7.500,00
Cámara web	Falabella	Chile	21.990,00	1	21.990,00
Sensor sonar Hc-sr04	Afel	Chile	2.300,00	1	2.300,00
Cables varios	MercadoLibre	Chile	63,50	100	6.350,00
Cable USB-micro USB	Falabella	Chile	3.990,00	1	3.990,00
Cable USB Arduino	Hubot	Chile	1.820,00	1	1.820,00
Cable USB tipo C	Falabella	Chile	3.990,00	1	3.990,00
Pack amarras plásticas para cables	MercadoLibre	Chile	16.646,00	1	16.646,00
				Total	616.033,00

Figura 23: Costos totales, parte eléctrica

3.3. Planilla de gastos realizados

Las siguientes tablas detallan los gastos realizados para desarrollar el proyecto, sin contar los componentes que ya estaban disponibles, ya sea en el laboratorio como pertenecientes a un



integrante del grupo, donde el costo total corresponde a \$ 139.270,00 pesos chilenos:

Producto	Tienda	País	Precio unitario	Unidades	Total
Madera MDF	Sodimac	Chile	9.990,00	1	9.990,00
Acoples rígidos (diferentes dimensiones)	Cimech3D	Chile	2.290,00	12	27.480,00
Eje de 8mm de diámetro y 1m de longitud	Cimech3D	Chile	12.990,00	1	12.990,00
Basurero	Ikea	Chile	3.990,00	1	3.990,00
Soportes con rodamiento	Cimech3D	Chile	3.290,00	4	13.160,00
				Total	67.610,00

Figura 24: Gastos totales, parte mecánica

Producto	Tienda	País	Precio unitario	Unidades	Total
Optocuplas PC817	Afel	Chile	2.990,00	7	20.930,00
Placas preperforadas	Afel	Chile	1.200,00	8	9.600,00
Pin header 40 pines (Macho/Hembra)	MCI	Chile	250,00	4	1.000,00
Regulador de Voltaje Step Down XH-M400	Afel	Chile	5.000,00	2	10.000,00
Servo-motor Mg995	Afel	Chile	7.000,00	2	14.000,00
Acelerómetro-giroscopio ADXL345	Afel	Chile	2.300,00	2	4.600,00
Cables varios	MercadoLibre	Chile	63,50	40	2.540,00
Cable USB tipo C	Falabella	Chile	3.990,00	1	3.990,00
Motor Ppaso a paso 28byj-48	Afel	Chile	2.500,00	2	5.000,00
				Total	71.660,00

Figura 25: Gastos totales, parte eléctrica

4. Manual de Usuario

4.1. Hardware y software necesario

Para iniciar el manual de usuario, se presentan los componentes necesarios para construir el prototipo:

Componentes mecánicos:

- 2 planchas de madera mdf 6mm 600 x 440 mm (una para cada piso).
- 2 acoples rígidos 5mm a 8mm.
- 2 acoples rígidos 8mm a 8mm.
- 4 acoples rígidos 6mm a 8mm.
- 4 Ruedas Juguete Monster Truck para arena.



- 4 pasadores con tuercas.
- 4 Ejes de 8mm de diámetro de 8cm de longitud.
- 2 Kg de material de impresión 3D PLA.
- 1 Basurero de 190mm de ancho.
- Tornillos y tuercas de diferentes diámetros.
- 4 Acoples tipo L para motores DC pololu.
- 4 Soportes con rodamiento.

Componentes eléctricos:

- ESP32 dev kit.
- Arduino MEGA.
- Raspberry Pi 4b.
- 6 Optocuplas PC817.
- 6 Placas preperforadas (11cm x 6cm, 6cm x 4cm, 7cm x 3cm x4)
- 12 Resistencias de $10k\Omega$.
- Pin header macho.
- Pin header hembra.
- 2 Reguladores de Voltaje Step Down XH-M400 8A.
- 2 Puentes H L298N.
- 4 Motores DC pololu 12V con encoder.
- 4 Servo-motores Mg995.
- 1 Acelerómetro-giroscopio ADXL345
- 2 Baterías Lipo 11.1V 2200 mAh.
- 1 Batería Lipo 14.8V 5200mAh.
- 3 Interruptores 2p 21x15mm.
- 2 Conector XT60 hembra.
- 1 Conector XT60U-F hembra.
- 1 Cámara web de resolución 640x480 píxeles.
- 1 Sensor sonar Hc-sr04.



- Cables jumper de conexión lógica y cables de transmisión de potencia.
- 2 Cables USB tipo Arduino.
- 1 Cable USB tipo C.
- Amarres para cables

Requerimientos de software, conocimientos y herramientas computacionales

- Arduino IDE
- 1 computador con entorno de programación para python (de de preferencia VScode).
- Programa de instalación de sistemas operativos (de preferencia Rufus).
- Sistema operativo de Raspberry pi.
- Librerías de python mínimas: Numpy, Serial, Open CV, Imutils, Time, Scipy.
- Librerías de Arduino mínimas: Simple_MPU6050, Servo.

Otras herramientas necesarias

- Impresora 3D.
- Cortadora láser CNC.
- Kit de llaves Allen.
- Kit de destornilladores.
- Alicates.
- Pistola de silicona.

4.2. Instalación, ensamblaje y configuración

Ensamblaje:

- Cortar las planchas de madera MDF con la cortadora láser, la primera plancha es cortada según el diseño presente en el archivo *Piso1_v1.dxf* y la segunda debe ser según el archivo *Piso2_v2.dxf*. Estas piezas corresponden a los pisos uno y dos del prototipo.
- Se deben imprimir los modelos 3D para carcasas de los componentes según los archivos *carcasa_motores.ipt* para los motores, *L298N_Case_1.0.3b.stl* y *L298N_Case_1.0.3c.stl* para los puentes H, *arduino mega case.ipt* para Arduino mega y *carcasa_baterias.ipt* y *carcasa_lippo.ipt* para las baterías.



- En la plancha correspondiente al primer piso, ensamblar los motores DC con sus respectivos soportes en forma de L. Luego, colocar un acople 6-8mm entre el eje del motor y un eje de 8mm, un rodamiento por motor y colocar dentro de su respectiva carcasa y atornillar.
- Colocar puentes H dentro de sus respectivas carcasas y atornillar entre motores (uno para dos motores).
- Con la pistola de silicona pegar tres optocuplas consiguientes a cada puente H. Las optocuplas deben ser dispuestas en paralelo desde los puentes H hacia el centro del chasis.
- Con las placas preperforadas, diseñar PCB's descritas en el anexo 7.3 y pegar cuidadosamente en el chasis como más acomode.
- Colocar arduino mega en su carcasa y ensamblar al chasis donde más acomode (preferentemente cerca de las optocuplas).
- Unir acoples con las ruedas y ensamblar al eje de cada motor.
- Colocar baterías en sus carcasas y ensamblar en la parte trasera del motor.
- Se ensambla el brazo con los respectivos servos y sonar dentro de este según los planos del brazo presentados en el diagrama de bajo nivel.
- Se colocan los servos de la base del brazo en sus respectivas carcasas y luego se acopla la base al brazo.
- Se ensambla el brazo al chasis.

Conexiones:

- Conectar a cada batería un interruptor on-off.
- Se debe conectar a la salida de una batería cuatro cables (dos alimentaciones y dos tierras que vayan conectadas a los dos puentes H (una alimentación y una tierra para cada puente H).
- Desde los puentes H se conectan seis cables que corresponden a señales (pines lógicos), estos van conectados a las optocuplas.
- Desde las optocuplas se conectan seis cables a una PCB (disponible en el anexo 7.3). Esta pcb recibe las señales desde las optocuplas y conecta con Arduino Mega a los pines correspondientes (disponibles en diagrama de bajo nivel).
- Conectar encoders de los motores a PCB de forma que quede cómodo conectar a la ESP32.
- La ESP32 recibe los encoders, un cable de alimentación y un cable de tierra.
- Una vez hechas las conexiones y ensamblado el segundo piso, se dispone la Raspberry pi y se conecta con el Arduino Mega por comunicación serial, de esta forma podremos enviar la información al Arduino para que controle los motores.



- Conectar cámara web a Raspberry pi por comunicación serial.

Instalación de software:

- Dirigirse al [sitio web oficial](#) de Raspberry Pi para descargar el sistema operativo.
- Descargar el OS llamado "Raspberry Pi OS (64-bit)".
- Descargar Rufus desde su [sitio web oficial](#)
- Abrir Rufus, seleccionar el sistema operativo descargado y conectar la memoria SD donde se va a realizar la partición. Iniciar el proceso de instalación.
- Luego de terminado el proceso, retirar la memoria SD e insertarla en la Raspberry Pi.
- Conectar la Raspberry Pi a un monitor por HDMI, conectar ratón y teclado. Luego, energizar la Raspberry.
- Seguir las instrucciones de instalación.
- Luego de instalado el OS, abrir un terminal y ejecutar los siguientes códigos:

```
$ pip3 install numpy
$ pip3 install scipy
$ pip3 install imutils
$ pip3 install opencv-python
```
- Transferir los archivos de extensión *.py* de la carpeta de códigos.
- Cargar el archivo *Wall2_MARK5.ino* al Arduino Mega y el archivo *Wall2_ESP32.ino* a la ESP32 a través de la IDE de Arduino.
- Conectar el Arduino Mega y la ESP32 a la Raspberry mediante conexión USB.

4.3. Instrucciones de uso

- Encender el interruptor que energiza la Raspberry Pi y el resto de controladores.
- Encender los interruptores que alimentan a los motores y los servos.
- Ejecutar el código "*Wall2_MARK5.py*"



5. Análisis de resultados finales

Como se ha mencionado en secciones anteriores en el informe, el desempeño del robot en la presentación final no fue el óptimo. Es por esta razón que, a continuación, se va a dedicar una sección exclusiva que resuma todos los avances más significativos del proyecto, los objetivos logrados y no logrados que se habían planteado en este proyecto, los problemas que se tuvieron a lo largo del proyecto y sus posibles causas y un apartado de trabajo futuro que señalen todos los aspectos a mejorar del robot.

5.1. Experimentos realizados

Para poder verificar el cumplimiento de los objetivos planteados al inicio del proyecto se realizaron experimentos de forma modular, dando paso a pruebas de una única funcionalidad, las que permitían paso a paso demostrar el funcionamiento del robot.

Segmentación de Mascarillas

Este experimento demuestra que el controlador era capaz de detectar la presencia o no de una mascarilla y, en el caso de que efectivamente hubiese una, segmentar y ubicar en el espacio la localización de ella. La demostración se encuentra en el archivo de grabación "*demo_segmentacion.mp4*", en el que se puede observar cómo el algoritmo logra segmentar de manera exitosa la mascarilla. Como observación, se puede notar que al voltear la mascarilla, la segmentación decae en desempeño, esto se debe a que los parámetros de la segmentación son determinados de forma heurística, por lo que en este caso el rango de colores no consideró tonos demasiado blancos.

PICK-UP Estático

Este experimento demuestra el adecuado funcionamiento del protocolo de instrucciones que hacen que el brazo robótico se mueva para recoger una mascarilla. Este experimento consiste en que el robot se sitúa frente a una mascarilla. Luego, se envía la instrucción de PICK-UP para recoger la mascarilla. El video de demostración se llama "*demo_pickup.mp4*".

Orientación respecto a mascarilla segmentada

Este experimento demuestra que, en base a la segmentación de imágenes y la ubicación del objeto en la cámara, el robot envía señales a sus ruedas para orientarlo, de modo que mire directamente hacia la mascarilla. El video con la demostración corresponde a "*demo_orientacion.mp4*".



Modo exploración

Este experimento tuvo como objetivo demostrar el protocolo de exploración implementado para WALL2. El resultado esperado es que el robot sea capaz de realizar una trayectoria cuadrada que representa el terreno a cubrir por el robot en dicho modo. El video de demostración corresponde a "*demo_trayectorias.mp4*". Como se puede apreciar en el video, el robot es capaz de mantener la trayectoria deseada, completando así el cuadrado. Sin embargo, es importante destacar que el experimento fue realizado en piso plano, por lo que no se garantiza que tenga el mismo comportamiento en la arena. Además, para demostrar que el algoritmo es robusto ante perturbaciones, el video "*demo_linea_recta.mp4*" muestra al robot siguiendo una trayectoria en línea recta, mientras es perturbado con fuerzas externas. Se puede apreciar como el robot logra retornar a la trayectoria establecida luego de haber sido movido.

Modo recolección

Este experimento tuvo como objetivo demostrar que el robot, al ya haber reconocido una mascarilla, es capaz de acercarse y recogerla. Este experimento se hizo asumiendo que el robot ya se había orientado correctamente, por lo que solo debía avanzar en línea recta. El video de demostración corresponde a "*demo_recoleccion.mp4*". En él, se puede notar como el robot se detiene en el momento en que se encuentra a una distancia ideal de la mascarilla, para posteriormente ejecutar el protocolo de PICK-UP. También es posible apreciar que el robot es capaz de recoger una serie de mascarillas, siempre y cuando estas se encuentren justo al frente de él.

5.2. Problemas, fallas críticas y sus causas

Luego de mostrado los avances más significativos en el proyecto propuesto, es necesario describir los diferentes problemas y fallos que sucedieron a lo largo del desarrollo del proyecto, los cuales provocaron retrasos de tiempos significativos.

Diseño del chasis del robot

Una de las etapas fundamentales que se debían completar cuanto antes corresponde al diseño del chasis, dado que este debe adecuarse a todos los componentes que va a poseer el sistema. A lo largo del semestre, el proyecto pasó por diferentes iteraciones en su diseño, entre ellos abarcan eliminación o incorporación de componentes, cambios en la posición de ensamblaje al chasis, aumentos en el espacio necesario, entre otros. Esto dio como resultado que se el chasis haya tenido un total de 5 versiones diferentes, lo que implica que se gastó una cantidad no menor de tiempo en volver a manufacturar el chasis, dado que la mayoría de pruebas no se podían realizar mientras el chasis no estaba listo.



Alineación de los ejes de los motores

Un momento crítico en el desarrollo con el proyecto fue un problema que surgió al momento de realizar el primer ensamblaje de los motores, los ejes y las ruedas. Al momento de hacer girar las ruedas en el aire (sin carga), se pudo notar que el consumo de corriente era superior a lo que se esperaba. Luego, al encender la base móvil para que se desplace en el piso, se pudo notar que los motores no eran capaces de mover el robot y que el consumo de corriente alcanzaba los límites que podía entregarla fuente de poder que los estaba alimentando en ese momento. Por esta razón, se tuvieron que suspender las pruebas hasta determinar la causa del problema y corregirlo. Al momento de dismantelar el chasis, se pudo descubrir el origen del problema, eje de la rueda se encontraba desalineado con respecto al eje del motor, lo que causaba que el eje no pudiera girar apropiadamente, obligando al motor a generar más torque del necesario para hacer girar el eje a la velocidad deseada, provocando un consumo de corriente excesivo, que repercutía en los puentes H y la fuente de alimentación. Para solucionar el problema, se tuvieron que diseñar piezas en 3D que compensaran la diferencia de alturas entre los ejes, lo cual fue una reparación que tomó aproximadamente una semana en concretarse.

Diseño del brazo robótico

Originalmente, el brazo robótico del proyecto utilizaba dos motores paso a paso 28byj-48, los cuales se encargaban de levantar el brazo completo. Acorde a los cálculos previos, estos motores eran capaces de generar el torque necesario para cumplir con su tarea. Al momento de fabricar el brazo en su totalidad, se realizó una prueba con dichos motores, para verificar su adecuado funcionamiento. Al comenzar la prueba, se observó que los motores no eran capaces de levantar el brazo. Para solventar rápidamente el problema, la base del brazo fue rediseñada, sustituyendo los motores paso a paso por servo-motores Mg995, los cuales eran capaces de generar más torque que los motores anteriores. De esta forma, se logra obtener el diseño final presentado anteriormente en el diagrama de bajo nivel. Dicho cambio detuvo el desarrollo por lo menos la mitad de una semana.

Pruebas de tracción en la arena

Una de las principales fallas en el diseño final es que, al momento de probar el robot en arena en la presentación final, el robot no era capaz de desplazarse en el terreno. La principal razón de este fallo fue la falta de pruebas en el terreno real, gran parte de los experimentos en los que se probaba la base móvil fueron realizados dentro del laboratorio, en los que se cuenta con piso plano, sin mayores irregularidades en el terreno, por lo que gran parte del diseño fue probado y ajustado en este ambiente y no en el real. Dentro de las pocas pruebas que se realizaron en arena, existe una en la que el robot lograba efectivamente desplazarse en el terreno. Sin embargo, en esa ocasión, se utilizó una versión incompleta del robot, el cual contaba solamente con los motores y las baterías instalados. Por lo tanto, este experimento, a pesar de ser exitoso, no considera el peso extra que tienen el resto de componentes que no habían sido agregados aún. Por otro lado, la forma del chasis final difiere en varios aspectos del chasis utilizado en ese experimento, lo que implica que el cambio en la geometría pudo haber perjudicado también en la tracción que genera la base móvil.



Sobrecalentamiento de un puente H

Los puentes H son los drivers que nos permiten controlar la dirección y velocidad de giro de las ruedas de *WALL2*. Durante las pruebas realizadas nos dimos cuenta de que el driver correspondiente a los motores traseros se sobre calentaba llegando a temperaturas muy superiores al driver correspondiente a los motores delanteros. Este problema terminó finalmente quemando 2 veces el puente H de las ruedas traseras.

Como grupo, analizando las posibles causas de este problema, llegamos a estas 2 posibles situaciones que generaron este problema:

- **Desajuste de las ruedas traseras**

Al ensamblar el diseño final de *WALL2* fijamos los motores al chasis y los cubrimos con sus respectivas protecciones. A medida que hicimos pruebas y hacíamos navegar al robot nunca reajustamos ni apretamos los tornillos de los motores. Esto resultó en que los motores traseros se soltaran, permitiendo juego entre los motores y el chasis, desalineando el eje del motor con el rodamiento que iba a las ruedas y generando peaks de corriente en dichos motores y sobre calentando de sobremanera el puente H, llegando a quemarlo.

- **Regulador de voltaje del puente H en mal estado**

Al revisar el puente H quemado nos dimos cuenta que el regulador de voltaje interno que tienen estos drivers estaba quemado, por lo que la salida que debiese ser de 5V era de 12V, generando que el circuito lógico del driver fuese alimentado con dicho voltaje, pudiendo generar el aumento de temperatura y posterior quema del chip.

Daño en la *ESP32*

El último problema que nos surgió fue que al momento de conectar la *ESP32* de forma definitiva, nos dimos cuenta de que su temperatura era extremadamente alta y que se había quemado.

Como grupo analizamos las posibles causas y la única conclusión posible a la que llegamos fue que a través de la conexión de las tierras haya existido una corriente parásita proveniente de la alimentación de los servomotores que haya producido que la *ESP32* se quemara. Esta corriente parásita se pudo haber dado ya que la alimentación de los servomotores y de los encoders vienen de un regulador de 7.2V, y al ser la *ESP32* el microcontrolador que lee los encoders la tierra de todo el circuito debe ser común.

5.3. Objetivos cumplidos y no cumplidos

Para este proyecto se plantearon 4 objetivos específicos los cuales, con el resultado final de nuestro robot, pudimos evaluar si fueron cumplidos o no. Estos son:

- **Diseñar e implementar base móvil con sistema de tracción capaz de movilizarse y maniobrar en arena sin dificultades.**



- Implementar sistema de pick-up y almacenamiento de mascarillas.
- Desarrollar un algoritmo de clasificación y detección de mascarillas con una visibilidad sobre la arena de un 50% para lograr su segregación y posterior recolección.
- Desarrollar protocolo de navegación sobre zona de la playa a limpiar.

Respecto al primer objetivo, sí fuimos capaces de implementar una base móvil con tracción, el cual se desplaza de buena manera en un suelo plano y sin muchas irregularidades. Ahora bien, hemos fallado en que este sistema de tracción sea apto para trasladarse en la arena. Esto se puede deber a varios factores. El primero de ellos es el gran peso que tuvo el robot en su diseño final. A medida en que se fue avanzando en su construcción, poco a poco se tuvo que ir aumentando su tamaño debido a que era necesario agregar nuevos componentes eléctricos, como, por ejemplo, las optocuplas PC817 o los reguladores de voltaje. Otro factor importante fue que no se hicieron pruebas suficientes en la arena cuando éste ya se encontraba con todo su peso final, lo que provocó que el robot se quedara atascado en la misma arena.

Por otro lado, el segundo objetivo se logró cumplir exitosamente. Luego de mucho tiempo de diseño del brazo robótico e intentar distintas maneras de afirmarlo al chasis, el brazo funciona de manera rápida y fluida, y es capaz de dejar caer las mascarillas en un recipiente destinado a su almacenamiento.

Ahora bien, sobre el tercer objetivo, hemos decidido que para detectar una mascarilla sea una segmentación por color, por lo que siempre que se vea de manera clara una parte de la mascarilla que se encuentre dentro del rango de los colores escogidos, entonces el algoritmo reconocerá a dicha mascarilla, lo que permite segmentar mascarillas que tengan una visibilidad menor al 50% sobre la arena. Pero el problema con esto es que si el robot se encuentra con una mascarilla que se vea por el otro lado, entonces la cámara no la reconocerá debido a su color. Por lo tanto, este objetivo no es el adecuado para describir realmente lo que terminó siendo el proyecto final.

Por último, respecto al último objetivo, sí se logró desarrollar un protocolo de navegación, y este protocolo utilizaba la información que entregaba los encoders de los motores DC para calcular la distancia recorrida, pero como se mencionó anteriormente, la ESP32 la cual estaba encargada de leer dichos encoders se quemó. Por lo que, debido a esto debimos cambiar el protocolo basándonos en el tiempo de ejecución del algoritmo, lo que no es recomendable, ya que puede haber casos en que el robot no avance y el tiempo sigue avanzando, afectando así el momento en que cambias las instrucciones que reciben los motores DC. Por lo tanto, sí logramos realizar un protocolo de navegación, pero éste tuvo que cambiarse para adaptarse a la situación que se vivió.

5.4. Mejoras al proyecto

Dado todo lo que se mencionó con anterioridad, es claro que se pueden hacer mejoras al proyecto. Primero, cambiar el sistema de tracción por uno en donde haya reducción con el fin de generar mejor tracción en la arena. Segundo, se debe mejorar la selección de ruedas, dado que estas



sufrieron complejidades al momento de desplazarse en arena, y además que sean capaces de traccionar considerando el peso final del robot. Por otro lado, es necesario implementar mayor protección a los componentes eléctricos, ya sea cubriendo los componentes con carcasas más herméticas o construyendo un chasis cerrado que impida el paso de material granulado cerca de los componentes.

Respecto a las mejoras eléctricas es fuertemente recomendable hacer las PCB's en una enrutadora, ya que es fácil que se produzcan cortocircuitos haciéndolas a mano como nos ocurrió durante el desarrollo del proyecto. Es necesario también la disposición de una Raspberry de mayor capacidad, esto dado que el robot requiere tomar datos en tiempo real para poder tomar decisiones.

En cuanto a las mejoras de software, se mencionó el uso de un algoritmo de segmentación por colores. Este método de reconocimiento puede ser mucho más preciso si se hace uso de redes neuronales convolucionales, las cuales darían al robot la posibilidad de recoger más de un tipo de mascarillas.

Por último, sabiendo que se tienen todos los componentes eléctricos y mecánicos necesarios, se debe intentar minimizar el peso del chasis para así realizar el menor esfuerzo mecánico posible y poder facilitar su traslación sobre la arena.



6. Referencias

- [1] Residuos Profesional, 2021. LOS RESIDUOS DE MASCARILLAS AUMENTARON UN 9.000% ENTRE MARZO Y OCTUBRE DE 2020. Disponible en el siguiente [LINK](#)
- [2] International Organization for Standardization. (2016). Robotics — Performance criteria and related test methods for service robots — Part 1: Locomotion for wheeled robots (ISO Standard No. 18646-1:2016). Disponible en el siguiente [LINK](#)
- [3] International Organization for Standardization. (2021). Robotics — Performance criteria and related test methods for service robots — Part 3: Manipulation (ISO Standard No. 18646-3:2021). Disponible en el siguiente [LINK](#)
- [4] Rainford Solutions, 2022. What is an IP Rating?: Ingress Protection. Disponible en el siguiente [LINK](#)
- [5] International Organization for Standardization. (2017). Mobile robots — Vocabulary (ISO Standard No. 19649:2017). Disponible en el siguiente [LINK](#)
- [6] Raspberry Pi, 2019. Raspberry Pi 4 model B, no.1. Disponible en el siguiente [LINK](#)
- [7] Arduino, 2022. Arduino® MEGA 2560 Rev3. Disponible en el siguiente [LINK](#)
- [8] Espressif Systems. (2019). ESP32 Series. Disponible en el siguiente [LINK](#)
- [9] Electronic Components Group SHARP Corporation, 1995. Device specification for Photocoupler model no. PC817. Disponible en el siguiente [LINK](#)
- [10] Pololu, 2022. 37D Metal Gearmotors. Disponible en el siguiente [LINK](#)
- [11] Marlin P. Jonse & Assoc., Inc. MG995 High Speed Servo Actuator. Disponible en el siguiente [LINK](#)
- [12] STMicroelectronics, 2000. DUAL FULL-BRIDGE DRIVER. Disponible en el siguiente [LINK](#)
- [13] InvenSense. MPU-6000 and MPU-6050 Product Specification. Disponible en el siguiente [LINK](#)



7. Anexos

7.1. Archivos

En la carpeta *Archivos/* Se encuentran las siguientes carpetas:

- *Brazo/*: Contiene todos los modelos necesarios para construir el brazo robótico.
- *Carcasas/*: Contiene los modelos de las protecciones de los componentes.
- *Codigos/*: Contiene los codigos que deben ser cargados a la Raspberry Pi, Arduino Mega y ESP32.
- *DiagramaPCB/*: Contiene los diagramas que se utilizaron para fabricar las tarjetas de interconexión.
- *Pisos/*: Contiene los archivos *.dxf* de corte laser para fabricar los pisos de la base móvil.
- *Videos/*: Contiene los videos de demostración, corresponden a los mismo que se enviaron por correo.

7.2. Cálculo de torque de los motores de las ruedas

En este caso, se asume que el robot diseñado se puede aproximar a una caja de masa uniformemente distribuida y masa total M , de modo que su centro de masa se encuentra en el centro geométrico de la caja. Luego, se define F como la fuerza de tracción que ejercen los motores para que el robot se mueva. Para adecuar mejor manera el robot al terreno en el cual va a desplazarse, se considera que el robot se encuentra en un plano inclinado con inclinación α y coeficiente de roce entre las ruedas y el suelo μ , de esta forma, se pretende simular los casos en que el robot deba desplazarse cuesta arriba, donde los motores deberán ejercer mayor torque. Por otro lado, el coeficiente de roce será utilizado para simular la resistencia que va a ejercer la arena en las ruedas del robot.

Por lo tanto, aplicando la ley de sumas de fuerzas, se tiene que:

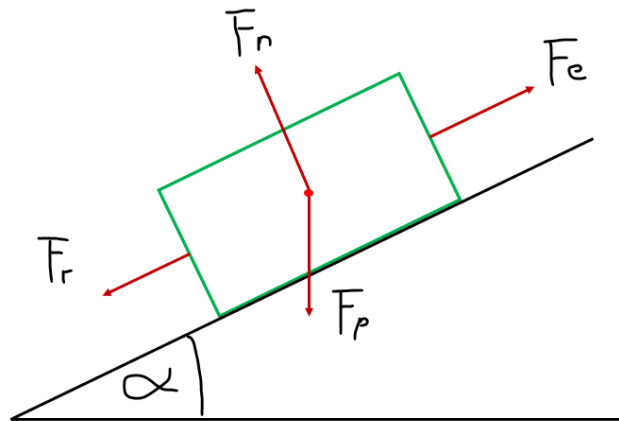


Figura 26: Diagrama de fuerzas del robot simplificado

$$\begin{aligned}\sum F &= Ma \\ F - F_r - Mg \cos\left(\frac{\pi}{2} - \alpha\right) &= Ma \\ F &= Ma + F_r + Mg \cos\left(\frac{\pi}{2} - \alpha\right) \\ F &= Ma + Mg \mu \cos(\alpha) + \sin(\alpha)\end{aligned}$$

Luego, el torque total que deben ejercer las ruedas del robot se calcula como:

$$\begin{aligned}\tau &= RF \\ \tau &= R(Ma + Mg \mu \cos(\alpha) + \sin(\alpha))\end{aligned}$$

Donde R corresponde al radio de las ruedas del robot. Para calcular el torque necesario, solo hace falta conocer el valor de la aceleración a . Para esto, se considera un perfil de velocidades trapezoidal, de velocidad máxima V_{max} , y tiempo de aceleración de t_{rise} . Entonces, la aceleración se calcula como:

$$a = \frac{V_{max}}{t_{rise}}$$

Finalmente, al considerar 4 motores para las ruedas del robot, el torque y velocidad angular necesario en los motores se pueden escribir como:

$$\begin{aligned}\tau &= \frac{R}{4} \left(M \frac{V_{max}}{t_{rise}} + Mg \mu \cos(\alpha) + \sin(\alpha) \right) \\ \omega &= \frac{V_{max}}{R}\end{aligned}$$



Cálculo de torque de los motores del brazo

En este caso, el brazo robótico se puede aproximar como dos objetos de masa uniformemente distribuida: un cilindro de largo L y masa M_1 , el cual tiene acoplado en uno de sus extremos una esfera de radio r y masa M_2 , que representa el sistema de pick-up. En este caso, el torque se puede escribir como:

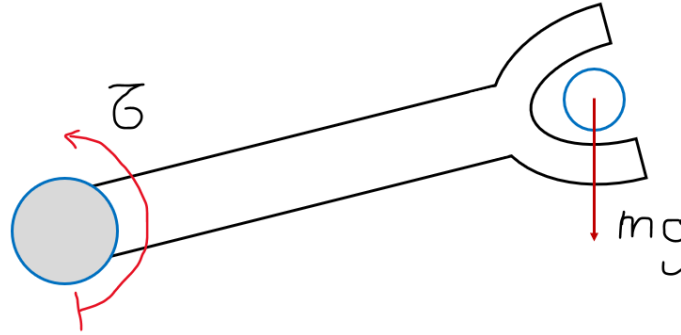


Figura 27: Diagrama de fuerzas del brazo simplificado

$$\tau = I_t \dot{\omega}$$

Donde el momento de inercia se escribe como:

$$I_t = I_1 + I_2 + M_2(L + r)^2$$

$$I_1 = \frac{1}{3} M_1 L^2$$

$$I_2 = M_2 r^2$$

Luego, la aceleración angular se calcula considerando un perfil de velocidades trapezoidal al igual que en el caso anterior, de modo que el cálculo es análogo:

$$\dot{\omega} = \frac{\omega_{max}}{t_{rise}}$$

Finalmente:

$$\tau = \frac{\omega_{max}}{t_{rise}} * \left(\frac{1}{3} M_1 L^2 + M_2 r^2 + M_2 (L + r)^2 \right)$$

Cálculo de torque de los motores del brazo

En este caso, el cálculo es similar al caso anterior, con la diferencia en que no se considera el brazo, solamente la esfera que representa el sistema de pick-up, por lo tanto:

$$\frac{\omega_{max}}{t_{rise}} * (M_2 r^2 + M_2 r^2)$$



7.3. Diagrama de tarjetas pre perforadas de interconexión

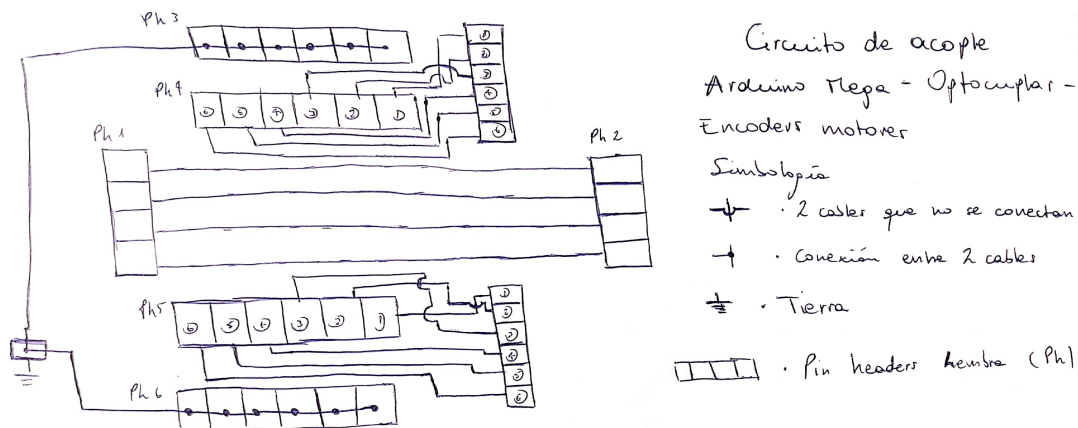


Figura 28: Diseño de PCB de conexión Arduino-Optocupla

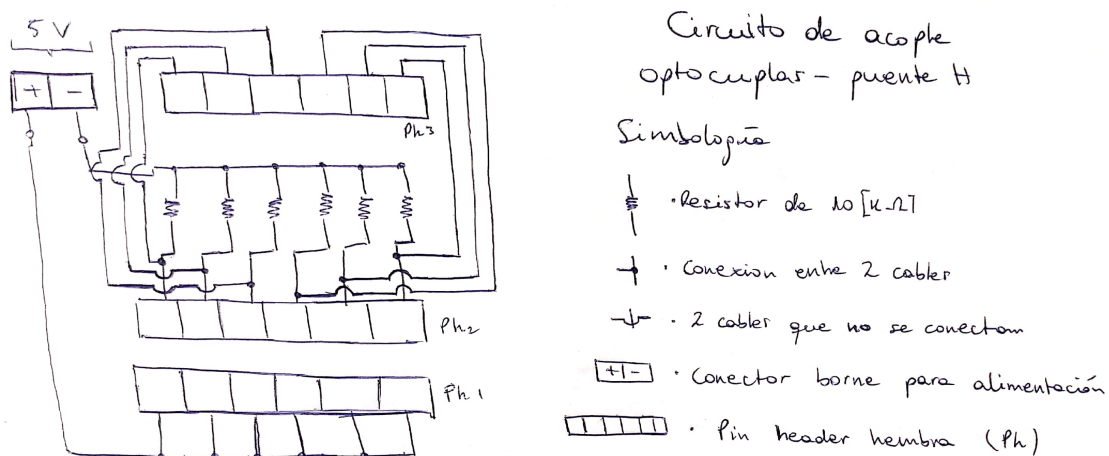


Figura 29: Diseño de PCB de conexión Optocupla-Puente H

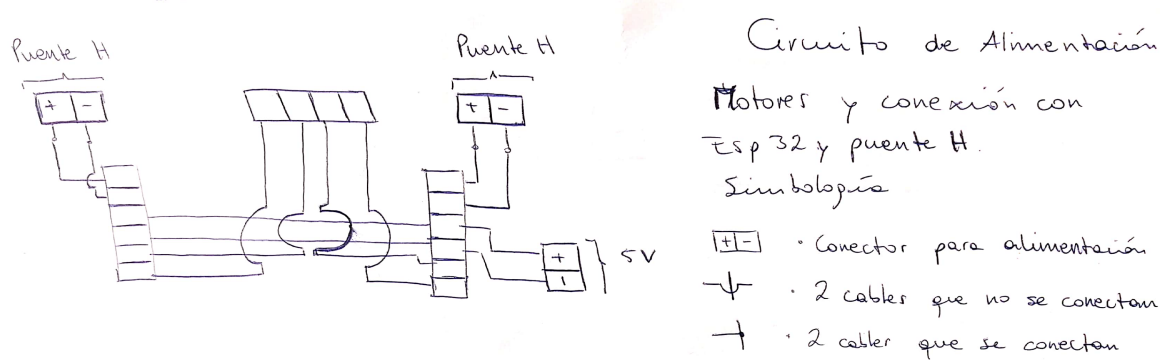


Figura 30: Diseño de PCB de conexión de motores